Best practices in Research Software Engineering

In this course we will discuss some tools and practices useful in software development (in particular, Research Software Engineering).

The material is derived from the CodeRefinery lessons (documentation and testing) and prepared for the NHR Summer School in Karlsruhe.

This material is divided in two parts:

- In the first part, we will see tools and best practices to write automated tests for our code, and to write and maintain its documentation
- In the second part, which tries to be as independent from the first as possible, we will see how to increase automation, by making use of infrastructure offered on software forges GitHub and GitLab.

⚙ Prerequisites

1. Basic understanding of Git.
2. For the Sphinx part, You need to have sphinx and sphinx_rtd_theme installed (they are part of the coderefinery environment).
3. You need `pytest` http://doc.pytest.org (as part of Anaconda or Miniconda or Virtual Environment).
4. (Optional) To work on exercises in other languages than Python, please follow the instructions under “Language-specific instructions” in the `Test design episode <https://coderefinery.github.io/testing/test-design/>` __ to install the recommended testing frameworks.

If you wish to follow in the terminal and are new to the command line, CodeRefinery has recorded a short shell crash course.

Part 1: Best practices in Research Software Engineering

Documentation

In this lesson we will discuss different solutions for implementing and deploying code documentation. This demonstration will be mostly **independent of programming languages** You will be able to follow and do all the exercises using the CodeRefinery conda environment).

We will then learn how to build documentation with the **documentation generator** Sphinx.

Then we will discuss the form of documentation which is the most immediate to write: **in-code** comments, and docstrings.

We will discuss best practices, and briefly discuss the available tools.

In-code documentation

We will step up our game and discuss **what makes a good README**. For many projects, a curated README is enough.

❓ Questions

environment).

We will then learn how to build documentation with the **documentation generator** Sphinx.

Then we will discuss the form of documentation which is the most immediate to write: **in-code** comments, and docstrings.

We will discuss best practices, and briefly discuss the available tools.

In-code documentation

We will step up our game and discuss **what makes a good README**. For many projects, a curated README is enough.

❓ Questions

- What can I do to make my code more easily understandable?
- What information should go into comments?
- What are docstrings and what information should go into docstrings?

In this episode we will learn how to write good documentation inside your code.

Demonstration - Writing good comments

✍ In-code-1: Comments

Let's take a look at two example comments (comments in Python start with `#`):

Comment A

```
# now we check if temperature is below -50
if temperature < -50:
    print("ERROR: temperature is too low")
```

Comment B

```
# we regard temperatures below -50 degrees as measurement errors
if temperature < -50:
    print("ERROR: temperature is too low")
```

Which of these comments is more useful? Can you explain why?

✔ Solution

- Comment A describes **what** happens in this piece of code. This can be useful for somebody who has never seen Python or a program, but for somebody who has, it can feel like a redundant commentary.
- Comment B is probably more useful as it describes **why** this piece of code is there, i.e. its **purpose**.

S

```
# do not run this code!
# if temperature > 0:
#     print("It is warm")
```

Keeping zombie code "just in case" (rather use version control):

Instead: Remove the code, you can always find it back in a previous version of your code in Git.

```
# do not run this code!
# if temperature > 0:
#     print("It is warm")
```

Keeping zombie code "just in case" (rather use version control):

Instead: Remove the code, you can always find it back in a previous version of your code in Git.

Emulating version control:

```
# John Doe: threshold changed from 0 to 15 on August 5, 2013
if temperature > 15:
    print("It is warm")
```

Instead: You can get this information from `git log` or `git show`, or `git blame` or similar.

What are "docstrings" and how can they be useful?

Here is function `fahrenheit_to_celsius` which converts temperature in Fahrenheit to Celsius, implemented in a couple of different languages. Your language is missing? Please contribute an example.

The first set of examples uses **regular comments**:

Python | R | Julia | Fortran | C++ | Rust

```
# This function converts a temperature in Fahrenheit to Celsius.
def fahrenheit_to_celsius(temp_f: float) -> float:
    temp_c = (temp_f - 32.0) * (5.0/9.0)
    return temp_c
```

The second set uses **docstrings or similar concepts**. Please compare the two (above and below):

Python | R | Julia | Fortran | C++ | Rust

```
def fahrenheit_to_celsius(temp_f: float) -> float:
    """
    Converts a temperature in Fahrenheit to Celsius.

    Parameters
    ----------
    temp_f : float
        The temperature in Fahrenheit.
```

```python
def fahrenheit_to_celsius(temp_f: float) -> float:
    """
    Converts a temperature in Fahrenheit to Celsius.

    Parameters
    ----------
    temp_f : float
        The temperature in Fahrenheit.

    Returns
    -------
    float
        The temperature in Celsius.
    """

    temp_c = (temp_f - 32.0) * (5.0/9.0)
    return temp_c
```

Read more: https://peps.python.org/pep-0257/

Docstrings can do a bit more than just comments:

- Tools can generate help text automatically from the docstrings.
- Tools can generate documentation pages automatically from code.

It is common to write docstrings for functions, classes, and modules.

Good docstrings describe:

- What the function does
- What goes in (including the type of the input variables)
- What goes out (including the return type)

Naming is documentation: Giving explicit, descriptive names to your code segments (functions, classes, variables) already provides very useful and important documentation. In practice you will find that for simple functions it is unnecessary to add a docstring when the function name and variable names already give enough information.

Keypoints

- Comments should describe the why for your code not the what.
- Writing docstrings can be a good way to write documentation while you type code since it also makes it possible to query that information from outside the code or to auto-generate documentation pages.

Writing good README files

For motivation: see MakeaREADME
The README file (often `README.md` or `README.rst`) is usually the first thing users/collaborators see when visiting your GitHub repository.

Demonstration: Have fun testing some README features

✍️ Exercise README-1: Have fun testing some README features you may not have heard about

Use it to communicate important information about your project! For many smaller or mid-size projects this is enough documentation. It's not that hard to make a basic one, and it's easy to expand as needed.

- Test the effect of adding the following to your GitHub README (read more):

For motivation: see MakeaREADME
The README file (often `README.md` or `README.rst`) is usually the first thing users/collaborators see when visiting your GitHub repository.

Demonstration: Have fun testing some README features

Use it to communicate important information about your project! For many smaller or mid-size projects, this is enough documentation. It's not that hard to make a basic one, and it's easy to expand as needed.

✍️ Exercise README-1: Have fun testing some README features you may not have heard about

- Test the effect of adding the following to your GitHub README (read more):

```
> [!NOTE]
> Highlights information that users should take into account, even when
skimming.

> [!IMPORTANT]
> Crucial information necessary for users to succeed.

> [!WARNING]
> Critical content demanding immediate user attention due to potential risks.
```

- For more detailed descriptions which you don't want to show by default you might find this useful (please try it out):

```
<details>
<summary>
Short summary
</summary>

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
</details>
```

- Would you like to add a badge like this one: 

?

  Badge that links to a website (see also https://shields.io/):

```
[![please replace with alt text](https://img.shields.io/badge/anytext-youlike-
blue)](https://example.org)
```

  Badge without link:

```
![please replace with alt text](https://img.shields.io/badge/anytext-youlike-
blue)
```

- Know about other tips and tricks? Please share them (send a pull request to this lesson).

- You can do that either by screensharing and discussing or working individually.
- Use the checklist which we have discussed earlier.
- Think about the user (**which can be a future you**) of your project, what does this user need to know to use or contribute to the project? And how do you make your project attractive to use or contribute to?
- (Optional): Try the https://hemingwayapp.com/ to analyse your README file and make your writing bold and clear.

Optional Exercise: Improve the README for your own project

✍️ Try to draft a brief README or review a README which you have written for one of your projects.

- You can do that either by screensharing and discussing or working individually.
- Use the checklist which we have discussed earlier.
- Think about the user (**which can be a future you**) of your project, what does this user need to know to use or contribute to the project? And how do you make your project attractive to use or contribute to?
- (Optional): Try the https://hemingwayapp.com/ to analyse your README file and make your writing bold and clear.
- Please note observations and recommendations in the collaborative notes.

Optional Exercise: Improve the README for your own project

✍ Try to draft a brief README or review a README which you have written for one of your projects.

Optional Exercise: Discuss the README of a project that you use

✍ Exercise README-3: Review and discuss a README of a project that you have used

In this exercise we will review and discuss a README of a project which you have used. You can also review a library which is popular in your domain of research and discuss their README.

- You can do that either by screensharing and discussing or working individually.
- When discussing other people's projects please remember to be respectful and constructive. The goal of this exercise is not to criticize other projects but to learn from other projects and to collect the aspects that you enjoyed finding in a README and to also collect aspects which you have searched for but which are sometimes missing.
- Please note observations and recommendations in the collaborative notes.

Table of contents in README files

- GitHub automatically generates a table of contents for README.md files.
- On GitLab you can generate a TOC in Markdown with:

```
[[_TOC_]]
```

- With RST you can generate a table of contents (TOC) automatically by adding:

```
.. contents:: Table of Contents
```

Sphinx and Markdown

Objectives

- Understand how static site generators build websites out of plain text files.
- Create example Sphinx documentation and learn some Markdown along the way.

Try to compare the source code and the result side by side.

We will take the first steps in creating documentation using Sphinx, and learn some MyST flavored Markdown syntax along the way.

Our goal in this episode is to build HTML pages locally on our computers.

This lesson is built with Sphinx

💬 Before we start, let us verify whether we have the software we need

You may need to activate your CodeRefinery conda environment we set up in the

Try to compare the source code and the result side by side.

We will take the first steps in creating documentation using Sphinx, and learn some MyST flavored Markdown syntax along the way.

Our goal in this episode is to build HTML pages locally on our computers.

This lesson is built with Sphinx

Before we start, let us verify whether we have the software we need

You may need to activate your CodeRefinery conda environment we set up in the installation instructions. This was covered as part of the installation instructions, but the most usual command to do this is:

```
$ conda activate coderefinery
```

Check whether Python is available (you should see a version; precise version is not so important):

```
$ python --version

Python 3.11.5
```

Check whether Sphinx is available (you should see a version; precise version is not so important):

```
$ sphinx-build --version

sphinx-build 5.3.0
```

Check whether the quickstart tool is available (you should see a version; precise version is not so important):

```
$ sphinx-quickstart --version

sphinx-quickstart 5.3.0
```

Check whether MyST parser is available (you should see no output):

```
$ python -c "import myst_parser"
```

If the above commands produce an error (**command not found** or **module not found** or ModuleNotFoundError), please follow our installation instructions. But please don't give up if you don't have these - the episodes after this one will work even without these tools.

Sphinx-1: Generate the basic documentation template

Create a directory for the example documentation, step into it, and inside generate the basic documentation template:

```
$ mkdir doc-example
$ cd doc-example
```

✍ Sphinx-1: Generate the basic documentation template

Create a directory for the example documentation, step into it, and inside generate the basic documentation template:

```
$ mkdir doc-example
$ cd doc-example
```

We create the basic structure of the project manually.

File/directory	Contents
conf.py	Documentation configuration file
index.md	Main file in Sphinx

Let's create the `index.md` with this content:

```
# Documentation example with Sphinx

A small example of how to use Sphinx and MyST
to create easily readable and aesthetically pleasing
documentation.

```{toctree}
:maxdepth: 2
:caption: Contents:
some-feature.md
```
```

Note that indentation and spaces play a role here.

We also create a `conf.py` configuration file, with this content:

```
project = 'Test sphinx project'
author = 'Alice, Bob'
release = '0.1'

extensions = ['myst_parser']

exclude_patterns = ['_build']
```

For more information about the configuration, see the Sphinx documentation.

Let's create the file `some-feature.md` (in Markdown format) which we have just listed in `index.md`:

```
# Some feature

## Subsection

Exciting documentation in here.
Let's make a list (empty surrounding lines required):

- item 1
```

```
# Some feature

## Subsection

Exciting documentation in here.
Let's make a list (empty surrounding lines required):

- item 1

  - nested item 1
  - nested item 2

- item 2
- item 3
```

We now build the site:

```
$ ls -1

conf.py
index.md
some-feature.md

$ sphinx-build . _build

... lots of output ...
build succeeded.

The HTML pages are in _build.

$ ls -1 _build

_sources
_static
genindex.html
index.html
objects.inv
search.html
searchindex.js
some-feature.html
```

Now open the file `_build/index.html` in your browser.

- Linux users, type:

  ```
  $ xdg-open _build/index.html
  ```

- macOS users, type:

  ```
  $ open _build/index.html
  ```

- Windows users, type:

  ```
  $ start _build/index.html
  ```

- If the above does not work: Enter `file:///home/user/doc-example/_build/index.html` in your browser (adapting the path to your case).

Hopefully you can now see a website. If so, then you are able to build Sphinx pages locally. This is useful to check how things look before pushing changes to GitHub or elsewhere.

- If the above does not work: Enter `file:///home/user/doc-example/_build/index.html` in your browser (adapting the path to your case).
- Windows users, type:

```
$ start _build/index.html
```

Hopefully you can now see a website. If so, then you are able to build Sphinx pages locally. This is useful to check how things look before pushing changes to GitHub or elsewhere.

Note that you can change the styling by adding the line

```
html_theme = "<my favorite theme>"
```

in `conf.py`. For instance you can use `sphinx_rtd_theme` to have the Read the Docs look (make sure the `sphinx_rtd_theme` python package is available first)

➡ See also

Sphinx also provides a tool called `sphinx-quickstart` that guides you in the creation of the project. The reason why we do not use it here is that it does not use Markdown by default, but a more complex format called ReStructured Text (ReST).

Exercise: Adding more Sphinx content

✍ Sphinx-2: Add more content to your example documentation

1. Add a entry below `some-feature.md` labeled `another-feature.md` (or a better name) to the `index.md` file.
2. Create a file `another-feature.md` in the same directory as the `index.md` file.
3. Add some content to `another-feature.md`, rebuild with `sphinx-build . _build`, and refresh the browser to look at the results.
4. Use the MyST Typography page as help.

Experiment with the following Markdown syntax:

- *Emphasized text* and **bold text**
- Headings:

```
# Level 1

## Level 2

### Level 3

#### Level 4
```

```
1. item 1
2. item 2
3. item 3
1. item 4
1. item 5
```

- Simple tables:

```
1. item 1
2. item 2
3. item 3
1. item 4
1. item 5
```

- Simple tables:

```
| No.  |  Prime |
| ---- | ------ |
| 1    |  No    |
| 2    |  Yes   |
| 3    |  Yes   |
| 4    |  No    |
```

- Code blocks:

````
The following is a Python code block:
```python
  def hello():
      print("Hello world")
```

And this is a C code block:
```c
#include <stdio.h>
int main()
{
    printf("Hello, World!");
    return 0;
}
```
````

- You could include an external file (here we assume a file called “example.py” exists; at the same time we highlight lines 2 and 3):

````
```{literalinclude} example.py
:language: python
:emphasize-lines: 2-3
```
````

- Math equations with LaTeX should work out of the box. Try this (result below):

````
This creates an equation:
```{math}
a^2 + b^2 = c^2
```

This is an in-line equation, {math}`a^2 + b^2 = c^2`, embedded in text.
````

Older versions of Sphinx

In some older versions, you might need to edit `conf.py` and add `sphinx.ext.mathjax`:

This creates an equation:

```
This is an in-line equation, {math}`a^2 + b^2 = c^2`, embedded in text.
```

This creates an equation:

Older versions of Sphinx

In some older versions, you might need to edit `conf.py` and add `sphinx.ext.mathjax`:

```
extensions = ['myst_parser', 'sphinx.ext.mathjax']
```

Exercise: Sphinx autodoc

✍ Sphinx-3: Auto-generating documentation from Python docstrings

1. Write some docstrings in functions and/or class definitions to a python module `multiply.py`:

```
def multiply(a: float, b: float) -> float:
    """
    Multiply two numbers.

    :param a: First number.
    :param b: Second number.
    :return: The product of a and b.
    """
    return a * b
```

2. In the file `conf.py` add `autodoc2` to the "extensions", and add the list `autodoc2_packages` which will mention `"multiply.py"`:

```
extensions = ['myst_parser', "autodoc2"]

autodoc2_packages = [
    "multiply.py"
]
```

If you already have extensions from another exercise, just add `"autodoc2"` to the existing list.

4. Add `apidocs/index` to the toctree in `index.md`.

```
```{toctree}
:maxdepth: 2
:caption: Contents:

...
apidocs/index
```

5. Re-build the documentation and check the "API r... section.

✍ Sphinx-4: Writing Sphinx content with Jupyter

1. For simplicity, create a text-based notebook files `flower.md` in the same directory as the `index.md` file. This file will be converted to a Jupyter notebook by the `myst_nb` Sphinx extension and then executed by Jupyter. Fill the file with the following
```

```
...
apidocs/index
```

✍️ Sphinx-4: Writing Sphinx content with Jupyter

1. For simplicity, create a text-based notebook files `flower.md` in the same directory as the `index.md` file. This file will be converted to a Jupyter notebook by the `myst_nb` Sphinx extension and then executed by Jupyter. Fill the file with the following content:

```
---
file_format: mystnb
kernelspec:
  name: python3
---
# Flower plot

```{code-cell} ipython3
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(1, 1, figsize=(5, 8), subplot_kw={"projection": "polar"})
theta = np.arange(0, 2 * np.pi, 0.01)
r = np.sin(5 * theta)
ax.set_rticks([])
ax.set_thetagrids([])
ax.plot(theta, r);
ax.plot(theta, np.full(len(theta), -1));
```
```

Note that there needs to be a title in the notebook (a heading starting with a single `#`), that will be used as an entry and link in the table of content. 2. In the file `conf.py` modify `extensions` to remove `"myst_parser"` and add `"myst_nb"` (you will get an error if you include both):

```
extensions = ["myst_nb"]
```

Note that MyST parser functionality is included in MyST NB, so everything else will continue to work as before.

3. List `flower` in the toctree in `index.md`.

```
```{toctree}
:maxdepth: 2
:caption: Contents:
...
flower.md
```
```

4. Re-build the documentation and check the “Flower” section.
5. Alternatively, you can directly add `.ipynb` files saved from Jupyter notebook or Jupyter lab. Just make sure to list it in the toctree in `index.md` with the correct path.

Good to know

- The `_build` directory is a generated directory and should not be part of the Git repository. We recommend to add `_build` to `.gitignore` to prevent you from accidentally adding files below `_build` to the Git repository.
- sphinx-autobuild provides a local web server that will automatically refresh your view every time you save a file - which makes writing and testing much easier.
- If you have problems, consider cleaning manually the `jupyter_execute` directory.

Good to know

4. Re-build the documentation and check the “Flower” section.
5. Alternatively, you can directly add `.ipynb` files saved from Jupyter notebook or Jupyter lab. Just make sure to list it in the toctree in `_index.md` with the correct path.

- The `_build` directory is a generated directory and should not be part of the Git repository. We recommend to add `_build` to `.gitignore` to prevent you from accidentally adding files below `_build` to the Git repository.
- sphinx-autobuild provides a local web server that will automatically refresh your view every time you save a file - which makes writing and testing much easier.
- If you have problems, consider cleaning manually the `jupyter_execute` directory.
- This is useful if you want to check the integrity of all internal and external links:

```
$ sphinx-build . -W -b linkcheck _build
```

References

- Sphinx documentation
- Sphinx + ReadTheDocs guide
- For more Markdown functionality, see the Markdown guide.
- For Sphinx additions, see Sphinx Markup Constructs.
- https://docs.python-guide.org/writing/documentation/

Keypoints

- Sphinx and Markdown is a powerful duo for writing documentation.
- In the next episode we will learn how to deploy the documentation to a cloud service and update it upon every `git push`.

Popular tools and solutions

Questions

- What tools are out there?
- What are their pros and cons?

Objectives

- Choose the right tool for the right reason.

Documentation Tools: comparison

Comparison of the tools for documentation we have discussed so far

Type	Convenient	Easy	Maintainabile	Searchable	Readable	LLM-friendly	Notes
in-code doc	✅✅	🟨	✅🟨	🟨	❌	✅🟨	❌for users
README	✅	✅	✅🟨	🟨	✅	✅	Mostly enough
HTML Latex Generators	🟨(?)	❌	❌🟨	🟨	✅ (?)	❌✅	✅Physics/Math, ❌copy/paste
Jupyter Widgets	🟨	🟨/❌	✅✅	🟨 (?)	✅	🟨	✅for validation ❌for programmers

What do we mean?

Type	Convenient	Easy	Maintainabile	Searchable	Readable	LLM-friendly	Notes
README	✅	✅	✅🟨	🟨	✅	✅	Typically enough
HTML Latex Generators	🟨(?)	❌	❌🟨	🟨	✅ (?)	❌✅	✅Physics/Math, powerful ❌copy/paste
Jupyter Myst	🟨	🟨/ ❌	✅✅	🟨 (?)	✅	🟨	✅for validation programmers

What do we mean?

- **Convenience**: for programmers who live in code.
- **Easiness**: how easy is is to contribute and set up?
- **Maintainability** is good for those tools that can be version-controlled along with the code. It is even better if it is easy to check automatically that the information is correct (*does the output of a snippet of code match what is shown in the docs?*)
- **Searchability:** How easy is it to find the information we need?
- **Readability**: Can the documentation be rendered in a way that makes it easy to read?
- **LLM-friendliness**: how easy is to feed this documentation to an LLM?

HTML static site generators

There are many tools generate documentation that can be viewed locally, or hosted on the web.

Here are some HTML static site generators, relevant in our communities. These tools offer some or all of these features:

- **API Reference generation**: source code is read, scan for docstrings and render them
- **Search**: they offer a "whole site" search feature (non trivial, when viewing only one page). (if you can download)
- **Validation**: check that the code snipped in the documentation match the real behaviour of the code.
- **Continuous checks**: regenerate automatically every time you save, so that you can catch errors early

Python | R | C/C++ | Fortran | Julia | Rust | Any

- Sphinx ← **this is how this lesson material is built**
  - Generate HTML/PDF/LaTeX from RST and Markdown (MyST)
  - Basically all Python projects use Sphinx but **Sphinx is not limited to Python**.
  - Read the docs hosts public Sphinx documentation for free!
  - **API Reference generation**: via autodoc or autoapi
  - **Search:**
    - limited, keyword-based client-side (Javascript that runs in browser)
    - Full-text server-side on Read the docs
  - **Validation**: via doctest
- Doxygen:
  - **API Reference generation**: has also support for Python
- MkDocs: A Markdown-first static site generator (with a vast system of plugins developed independently).
  - **API Reference generation**: via mkdocstrings
  - **Search:** search plugin for client-side (Javascript that runs in the browser - lunr.js) Project now (as of 2026) not maintained [1]

- Doxygen:
  - **Validation**: via doctest
  - **API Reference generation**: has also support for Python
- MkDocs: A Markdown-first static site generator (with a vast system of plugins developed independently).
  - **API Reference generation**: via mkdocstrings
  - **Search:** search plugin for client-side (Javascript that runs in the browser - lunr.js) Project now (as of 2026) not maintained [1]

GitHub, GitLab, and Bitbucket make it possible to serve HTML pages:

- GitHub Pages
- Bitbucket Pages
- GitLab Pages

Read The Docs is also free to use for open source code, and can be connected to common software forges.

💬 Discussion

Do you know an awesome tool or feature that should be in this list? Let us know! (Open a PR)

Plain Text formats: reStructuredText and Markdown

```
# This is a section in Markdown   This is a section in RST
                                  ========================

## This is a subsection           This is a subsection
                                  --------------------

Nothing special needed for        Nothing special needed for
a normal paragraph.               a normal paragraph.

                                  ::

    This is a code block          This is a code block


**Bold** and *emphasized*.        **Bold** and *emphasized*.

A list:                           A list:
- this is an item                 - this is an item
- another item                    - another item

There is more: images,            There is more: images,
tables, links, ...                tables, links, ...
```

- Two of the most popular lightweight markup languages.
- reStructuredText (RST) has more features than Markdown but the choice is a matter of taste.
- Nice resource to learn Markdown: Learn Markdown in 60 seconds
- There are (unfortunately) many flavors of Markdown.
- Pandoc can convert between MD and RST (and many other formats).
- Motivation to stick to a standard text-based format: **They make it easier to move the documentation to other tools which also expect a standard format, as the project/organization grows**.

❗ Keypoints

- READMEs are typically a good starting point.
- We use MyST flavored Markdown in the Sphinx and Markdown episode and the Hosting websites/homepages on GitHub Pages example.
- Some popular solutions make reproducibility and maintenance of multiple code

- Nice resource to learn Markdown: Learn Markdown in 60 seconds
- There are (unfortunately) many flavors of Markdown.
- Pandoc can convert between MD and RST (and many other formats).
- Motivation to stick to a standard text-based format: **They make it easier to move the documentation to other tools which also expect a standard format, as the project/organization grows.**
- READMEs are typically a good starting point.
- Some popular solutions make reproducibility and maintenance of multiple code versions difficult.
- The landscape of tools is very diversified and every community has their own favourite.
- The basic functionality of all Static site generators is very similar, but specific aspects (API ref generation, search, validation) differ.

Keypoints

taste
- We use MyST flavored Markdown in the Sphinx and Markdown episode and the Hosting websites, Homepages on GitHub Pages example.

---

[1] After somewhat dramatic events (2026), MkDocs 1.x is now superseded by Zensical, which tries to keep compatibility wiht MkDocs 1.x. (MkDocs 2.0 is also being developed but projects and plugins based on 1.x will break).

Motivation and wishlist

Motivation

Motivation-1: Why documenting code?

Use the collaborative document:

- Is project documentation important? Why?
- How would you describe a useful documentation?
- How can you motivate your colleagues to contribute to the documentation?

Our motivation (but let us brainstorm first)

- You will probably use your code in the future and may forget details.
- You may want others to use your code (almost impossible without documentation).
- You may want others to contribute to the code.
- Shield your limited time and let the documentation answer FAQs.

What do we expect from a suitably good documentation?

Note

Documentation comes in different forms - what *is* documentation?

- **Tutorials**: learning-oriented, allows the newcomer to get started
- **How-to guides**: goal-oriented, shows how to solve a specific problem
- **Explanation**: understanding-oriented, explains a concept
- **Reference**: information-oriented, describes the machinery

Read more:

- https://www.writethedocs.org/guide/writing/beginners-guide-to-docs/
- https://www.divio.com/blog/documentation/
- https://diataxis.fr/

There is no one size fits all: often for small projects a `README.md` or `README.rst` can be enough (more about these formats later).

Creating a checklist

- **Explanation**: understanding-oriented, explains a concept
- **Reference**: information-oriented, describes the machinery

Read more:

- https://www.writethedocs.org/guide/writing/beginners-guide-to-docs/
- https://www.divio.com/blog/documentation/
- https://diataxis.fr/

There is no one size fits all: often for small projects a `README.md` or `README.rst` can be enough (more about these formats later).

Creating a checklist

💬 Motivation-2: Create a wishlist

Use the collaborative document:

- Let us create a wishlist for how we would like documentation to be.
- Below are some of our ideas but please do not look at them yet.
- We are sure you will come up with ideas we did not think about.

✔ Our wishlist (but let us brainstorm first)

Versions

- Your code project should be versioned (version control).
- Enable reproducibility and avoid confusion: **documentation should be versioned** as well.
- Have you ever seen: *"We will soon release a new version and are updating the documentation. Some features may not be available in the version you have downloaded."*?

Documentation should be placed and tracked close to the source code

- Documenting **close to the source code** (e.g. subdirectory `doc/`) minimizes barrier to contribute.
- I should not need to log in to another machine or service and jump through hoops to contribute.
- It is often good enough to have a `README.md` or `README.rst` along with your code/script.

Use a standard markup language

Markup

Markup is a set of human readable instructions that is used to tell the computer how a document shall be styled and structured. By using a markup language we can for example write a `*` or `-` where we want a bullet point to appear in the rendered document.

- offers formatting flexibility, enforces a basic document structure and the rendered documents can be exported to other formats (e.g. for printing). Also, the source can be read by humans without knowledge of the language in case the rendered document is unavailable.
- PDF alone is not enough since **copy-pasting out of a PDF document can be difficult**.
- It is OK to provide a generated PDF in addition to a copy-paste-able format.
- We suggest to use either reStructuredText (RST) or Markdown markup.
- GitHub and GitLab automatically render `README.md` or `README.rst` files.

Written by humans

Copy-paste-able

- Automatically generated documentation (e.g. API documentation) is useful as complementary documentation but it does not replace tutorials written by humans.

- read by humans without knowledge of the language in case the rendered document is unavailable.
- PDF alone is not enough since **copy-pasting out of a PDF document can be difficult**.
- It is OK to provide a generated PDF in addition to a copy-paste-able format.
- We suggest to use either reStructuredText (RST) or Markdown markup.
- GitHub and GitLab automatically render `README.md` or `README.rst` files.

Written by humans

Copy-paste-able

- Automatically generated documentation (e.g. API documentation) is useful as complementary documentation but it does not replace tutorials written by humans.

Installation instructions

- Give **step by step instructions for the basic case**. Additional information and caveats can be linked from there.
- List requirements and dependencies (libraries, compilers, environment).
- Include instructions for how to test for correctness after installation.

Make the license explicit

- **Include a LICENSE file** with your source code.
- Without a license, your work is under exclusive copyright by default: others are not allowed to re-use or modify anything.
- GitHub and GitLab allows to choose a license from common license templates.

Information for contributors

- Make it easy for others to contribute: **document how you prefer others to contribute**.
- Users of your code may be shy to contribute code. Your **documentation provides a platform for your first contributions**.

❗ Documentation checklist

Which items to include depends on the number of users apart from yourself.

- Purpose
- Authors
- License
- Recommended citation
- Copy-paste-able example to get started
- Dependencies and their versions or version ranges
- Installation instructions
- Tutorials covering key functionality
- Reference documentation (e.g. API) covering all functionality
- How do you want to be asked questions (mailing list or forum or chat or issue tracker)
- Possibly a FAQ section
- Contribution guide

Summary

❗ Keypoints

- Documentation is part of the code and should be versionable.
- Documentation (sources) should be tracked with the corresponding code in the same repository.
- Use standard markup languages such as reStructuredText or Markdown.

❓ Questions

- What recommendations can we take home?

Summary

Keypoints

- Documentation is part of the code and should be versionable.
- Documentation (sources) should be tracked with the corresponding code in the same repository.
- Use standard markup languages such as reStructuredText or Markdown.

❓ Questions

- What recommendations can we take home?

There is not the one right way: it is always a balance

Jupyter notebooks can be good documentation for scripts

- For simple scripts and post-processing, Jupyter notebooks can form a nice self-documenting pipeline.
- They can be a nice way to accompany a paper that analyzed some data.

READMEs or Sphinx?

- For smaller projects READMEs can be absolutely enough.
- If the code is closed-source (and hence nobody can see the README), you probably prefer Sphinx (or similar).
- If you need math equations, Sphinx might be a good fit.

How to make sure that code changes come together with documentation changes?

- Make documentation part of your code review.

Read the Docs or GitHub pages or both?

- GitHub pages typically serves one version (one branch). However, it is possible to build several or all branches as part of a workflow.
- Read the Docs can serve several versions (several branches/tags) at the same time.
- Some projects use both.

Consider making your development tutorial-driven

- Writing documentation is as important as writing software.
- Focus on how you use the software.
- If there is no tutorial on it, the feature "doesn't exist".
- Don't keep tutorial in sync with code, keep code in sync with tutorial - change the tutorial first.
- Read more in this fantastic slide-deck about tutorial-driven development.

Testing

Have you ever had some of these problems?:

- You change B and C, and suddenly A doesn't work anymore. Time wasted trying to figure out what changed.
- There was some simple problem, systematically testing could have found it.
- You get someone else's code and are afraid to touch it because who knows what might break. Plot twist: it's your own code!

People have learned that some automatic way to check problems makes software development much easier.

This part of the lesson will talk about the places it's useful for research code, and how easy it can be. We will discuss why testing often needs to be part of the software development cycle and how such a cycle can be implemented. We will see how automated testing works and practice designing and writing tests.

- You change B and C and suddenly A doesn't work anymore. Time wasted trying to figure out what changed.
- There was some simple problem systematically testing could have found it.
- You get someone else's code and are afraid to touch it because who knows what might break. Plot twist: it's your own code!

People have learned that some automatic way to check problems makes software development much easier.
This part of the lesson will talk about the places it's useful for research code, and how easy it can be. We will discuss why testing often needs to be part of the software development cycle and how such a cycle can be implemented. We will see how automated testing works and practice designing and writing tests.

Why automated testing?

Objectives

- Appreciate the importance of testing software
- Understand various benefits of testing

Untested software can be compared to uncalibrated detectors

"Before relying on a new experimental device, an experimental scientist always establishes its accuracy. A new detector is calibrated when the scientist observes its responses to known input signals. The results of this calibration are compared against the expected response."

> [From Testing and Continuous Integration with Python, created by K. Huff]

With testing, simulations and analysis using software *can* be held to the same standards as experimental measurement devices!

What can go wrong when research software has bugs? Look no further:

- A Scientist's Nightmare: Software Problem Leads to Five Retractions
- Researchers find bug in Python script may have affected hundreds of studies

---

Testing in a nutshell

In software tests, expected results are compared with observed results in order to establish accuracy. Why are we not comparing directly all digits with the expected result?:

Python | C++ | R | Julia | Fortran

```python
def fahrenheit_to_celsius(temp_f):
    """Converts temperature in Fahrenheit
    to Celsius.
    """
    temp_c = (temp_f - 32.0) * (5.0/9.0)
    return temp_c

# This is the test function: `assert` raises an error if something
# is wrong.
def test_fahrenheit_to_celsius():
    temp_c = fahrenheit_to_celsius(temp_f=100.0)
    expected_result = 37.777777
    assert abs(temp_c - expected_result) < 1.0e-6
```

```
$ python3 run-test.py
```

```
    # is wrong.
    def test_fahrenheit_to_celsius():
        temp_c = fahrenheit_to_celsius(temp_f=100.0)
        expected_result = 37.777777
        assert abs(temp_c - expected_result) < 1.0e-6
```

```
$ python3 run-test.py

running: sample_data/set1.csv --output=tests/set1.txt
CORRECT
```

What can tests help you do?

Preserving expected functionality

- Check old things when you add new ones

Help users of your code

- Verify it's installed correctly and works.
- See examples of what it should do.

Help other developers modify it

- Change things with confidence that nothing is breaking.
- Warning if documentation/examples go out of date.

Manage complexity

- If code is easy to test, it's probably easier to maintain.
- The next lesson Modular code development demonstrates this.

Discussion: When is it OK not to add tests?

💬 Discussion: When is it OK not to add tests?

Vote in the notes and we'll discuss soon. **It is always a balance: there is no "always"/"never".**

1. Jupyter or R Markdown notebook which produces a plot and you know by looking at the plot whether it worked?
2. A short, "obviously correct" Python or R script which you never intend to reuse?
3. A simple short, "obviously correct" shell script?
4. Can you give other examples?

Use the collaborative notes to answer these questions:

1. Give examples of things (from your work) that are easy to test.
2. Give examples of things (from your work) that are hard to test.

Discussion: What's easy and hard to test?

💬 Discussion: Testing in practice

Use the collaborative notes to answer these questions:

1. Give examples of things (from your work) that are easy to test.
2. Give examples of things (from your work) that are hard to test.

Discussion: What's easy and hard to test?

💬 Discussion: Testing in practice

Testing vocabulary

- Test functions one at a time - **Unit tests**
- Test how parts work together - **Integration tests**
- Test the whole thing running - **End-to-end tests**
  - For example, running on sample data.
- Check results are the same as before - **Regression tests**
- Write test first (the output), then write code to make test pass - **Test-driven development**
- GitHub or GitLab runs tests automatically - **Continuous integration**
- Report that tells you which lines were/were not run by tests - **Code coverage**
- Framework that runs test for you - **Testing framework**
  - See Quick Reference for some examples.

What should you do?

- Not every code needs perfect test coverage.
- If code is interactive-only (Jupyter Notebook), it's usually hard to test.
  - But also hard to run: the next lesson will discuss!
- At least end-to-end is often easy to add.
- Add tests of tricky functions.
  - If you'd have to run it over and over to test while writing, why not make it a property test?
- It's easy to have Gitlab/Github run the tests.
  - It's nice to push without thinking, and the system tells you when it's broke.
- **Learning how to test well make the rest of your code better, too.**

Where to start

- A simple script or notebook probably does not need an automated test.

If you have nothing yet

- Start with an end-to-end test.
- Describe in words how *you* check whether the code still works.
- Translate the words into a script.
- Run the script automatically on every code change.

❓ Questions

If you want to start with unit-testing

- How hard is it to set up a test suite for a first unit test?
- You want to rewrite a function? Start adding a unit test right there first.

Testing locally

In this exercise we will make a simple function and use one of the language specific test

❓ Questions

If you want to start with unit-testing

- How hard is it to set up a test suite for a first unit test?
- You want to rewrite a function? Start adding a unit test right there first.

Testing locally

In this exercise we will make a simple function and use one of the language specific test frameworks to test it.

- This is easy to use by almost any project and doesn't rely on any other servers or services.
- The downside is that you have to remember to run it yourself.

✍️ Local-1: Create a minimal example (15 min)

In this exercise, we will create a minimal example using the pytest, run the test, and show what happens when a test breaks.

1. Create a new directory and change into it:

```
$ mkdir local-testing-example
$ cd local-testing-example
```

2. Create an example file and paste the following code into it

Python | R | Julia | C++

Create `example.py` with content

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add('space', 'ship') == 'spaceship'
```

This code contains one genuine function and a test function. `pytest` finds any functions beginning with `test_` and treats them as tests.

3. Run the test

Python | R | Julia | C++

```
$ pytest -v example.py

============================================================ test session
starts ================================
platform linux -- Python 3.7.2, pytest-4.3.1, py-1.8.0, pluggy-0.9.0 --
/home/user/pytest-example/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/user/pytest-example, inifile:
```

```
$ pytest -v example.py

============================================================ test session
starts ================================
platform linux -- Python 3.7.2, pytest-4.3.1, py-1.8.0, pluggy-0.9.0 --
/home/user/pytest-example/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/user/pytest-example, inifile:
collected 1 item

example.py::test_add PASSED

============================================================ 1 passed in 0.01
seconds ================================
```

Yay! The test passed!

Hint for participants trying this inside Spyder or IPython: try `!pytest -v example.py`.

4. Let us break the test!

Introduce a code change which breaks the code (e.g. `-` instead of `+`) and check whether our test detects the change:

Python | R | Julia | C++

```
$ pytest -v example.py

============================================================ test session
starts ================================
platform linux -- Python 3.7.2, pytest-4.3.1, py-1.8.0, pluggy-0.9.0 --
/home/user/pytest-example/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/user/pytest-example, inifile:
collected 1 item

example.py::test_add FAILED

================================================================ FAILURES
=====================================
_____________________________________________________________ test_add
_____________________________________

    def test_add():
>       assert add(2, 3) == 5
E       assert -1 == 5
E         --1
E         +5

example.py:6: AssertionError
============================================================ 1 failed in 0.05
seconds ==============
```

Notice how pytest is smart and includes context: lines that failed, values of the relevant variables.

learn how to compare floating point numbers since they are more tricky (see also "What Every Programmer Should Know About Floating-Point Arithmetic").

Notice how pytest is smart and includes context: lines that failed, values of the relevant variables.

learn how to compare floating point numbers since they are more tricky (see also "What Every Programmer Should Know About Floating-Point Arithmetic").

The following test will fail and this might be surprising. Try it out:

Python | R | Julia | C++

```
def add(a, b):
    return a + b

def test_add():
    assert add(0.1, 0.2) == 0.3
```

Your goal: find a more robust way to test this addition.

✔ Solution: Local-2

Python | R | Julia | C++

One solution is to use pytest.approx:

```
from pytest import approx

def add(a, b):
    return a + b

def test_add():
    assert add(0.1, 0.2) == approx(0.3)
```

But maybe you didn't know about pytest.approx: and did this instead:

```
def test_add():
    result = add(0.1, 0.2)
    assert abs(result - 0.3) < 1.0e-7
```

This is OK but the `1.0e-7` can be a bit arbitrary.

functions beginning with `test_` for `pytest`.

- Python, Julia and C/C++ have better tooling for automated tests than Fortran and you can use those also for Fortran projects (via `iso_c_binding`).

Keypoints

Test design

This is OK but the `1.0e-7` can be a bit arbitrary.

- functions beginning with `test_` for `pytest`.
- Python, Julia and C/C++ have better tooling for automated tests than Fortran and you can use those also for Fortran projects (via `iso_c_binding`).

Keypoints

Test design

Questions

- How can different types of functions and classes be tested?
- How can the integrity of a complete program be monitored over time?
- How can functions that involve random numbers be tested?

In this episode we will consider how functions and programs can be tested in programs developed in different programming languages.

Exercise instructions

For the instructor

- First motivate and give a quick tour of all exercises below (10 minutes).
- Emphasize that the focus of this episode is *design*. It is OK to only discuss in groups and not write code.

During exercise session

- Choose the exercise which interests you most. There are many more exercises than we would have time for.
- Discuss what testing framework can be used to implement the test.
- Keep notes, questions, and answers in the collaborative document.

Once we return to stream

- Discussion about experiences learned.

Language-specific instructions

Python | C++ | R | Julia | Fortran

The suggested solutions below use pytest. Further information can be found in the Quick Reference.

Pure and impure functions

Python | C++ | R | Julia | Fortran

Start thinking how you would design tests for the following five functions, and then try to implement them. Also discuss why some are easier to test than others.

```
def factorial(n):
    """
    Computes the factorial of n.
    """
    if n < 0:
```

St Python ing how you would design tests for the following five functions, and then try to s. Also discuss why some are easier to test than others.

```python
def factorial(n):
    """
    Computes the factorial of n.
    """
    if n < 0:
        raise ValueError('received negative input')
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Discussion point: The factorial grows very rapidly. What happens if you pass a large number as argument to the function?

✔ Solution

This is a **pure function** so is easy to test: inputs go to outputs. For example, start with the below, then think of some what extreme cases/boundary cases there might be. This example shows all of the tests as one function, but you might want to make each test function more fine-grained and test only one concept.

Python

```python
import pytest

def test_factorial():
    assert factorial(0) == 1
    assert factorial(1) == 1
    assert factorial(2) == 2
```

Notes on the discussion point: Programming languages differ in the way they deal with integer overflow. Python automatically converts to the necessary `long` type, in Julia you would observe a “wrap-around”, in C/C++ you get undefined behaviour for signed integers. Testing for overflow likewise depends on the language.

✍ Design-2: Design a test for a function that receives two strings and returns a number

Python

```python
def count_word_occurrence_in_string(text, word):
    """
    Counts how often word appears in text.
    Example: if text is "one two one two three four"
             and word is "one", then this function returns 2
    """
    words = text.split()
    return words.count(word)
```

```python
def count_word_occurrence_in_string(text, word):
    """
    Counts how often word appears in text.
    Example: if text is "one two one two three four"
             and word is "one", then this function returns 2
    """
    words = text.split()
    return words.count(word)
```

✔ Solution

This is again a **pure function** but uses strings. Use a similar strategy to the above.

Python | C++ | R | Julia | Fortran

```python
def test_count_word_occurrence_in_string():
    assert count_word_occurrence_in_string('AAA BBB', 'AAA') == 1
    assert count_word_occurrence_in_string('AAA AAA', 'AAA') == 2
    # What does this last test tell us?
    assert count_word_occurrence_in_string('AAAAA', 'AAA') == 1
```

✍️ Design-3: Design a test for a function which reads a file and returns a number

Python | C++ | R | Julia | Fortran

```python
def count_word_occurrence_in_file(file_name, word):
    """
    Counts how often word appears in file file_name.
    Example: if file contains "one two one two three four"
             and word is "one", then this function returns 2
    """
    count = 0
    with open(file_name, 'r') as f:
        for line in f:
            words = line.split()
            count += words.count(word)
    return count
```

✔ Solution

Python | C++ | R | Julia | Fortran

e we test a function which is not pure, because the output depends on file. We can generate a temporary file for testing and remove it

```python
import tempfile
import os

def test_count_word_occurrence_in_file():
    _, temporary_file_name = tempfile.mkstemp()
```

Python C++ R Julia Fortran

e we test a function which is not pure, because the output depends on file. We can generate a temporary file for testing and remove it

```
import tempfile
import os

def test_count_word_occurrence_in_file():
    _, temporary_file_name = tempfile.mkstemp()
    with open(temporary_file_name, 'w') as f:
        f.write("one two one two three four")
    count = count_word_occurrence_in_file(temporary_file_name, "one")
    assert count == 2
    os.remove(temporary_file_name)
```

✍ Design-4: Design a test for a function with an external dependency

This one is not easy to test because the function has an external dependency.

Python C++ R Julia Fortran

```
def check_reactor_temperature(temperature_celsius):
    """
    Checks whether temperature is above max_temperature
    and returns a status.
    """
    from reactor import max_temperature
    if temperature_celsius > max_temperature:
        status = 1
    else:
        status = 0
    return status
```

✔ Solution

This function depends on the value of `reactor.max_temperature` so the function is not pure, so testing gets harder. You could use monkey patching to override the value of `max_temperature`, and test it with different values. Monkey patching is the concept of artificially changing some other value.

A better solution would probably be to rewrite the function.

Python C++ R Julia Fortran

```
def test_set_temp(monkeypatch):
    monkeypatch.setattr(reactor, "max_temperature", 100)
    assert check_reactor_temperature(99)  == 0
    assert check_reactor_temperature(100) == 0   # boundary cases easily go
wrong
    assert check_reactor_temperature(101) == 1
```

```python
def test_set_temp(monkeypatch):
    monkeypatch.setattr(reactor, "max_temperature", 100)
    assert check_reactor_temperature(99)  == 0
    assert check_reactor_temperature(100) == 0   # boundary cases easily go
wrong
    assert check_reactor_temperature(101) == 1
```

Python C++ R Julia Fortran

```python
class Pet:
    def __init__(self, name):
        self.name = name
        self.hunger = 0
    def go_for_a_walk(self):  # <-- how would you test this function?
        self.hunger += 1
```

✔ Solution

Python C++ R Julia Fortran

```python
def test_pet():
    p = Pet('fido')
    assert p.hunger == 0
    p.go_for_a_walk()
    assert p.hunger == 1

    p.hunger = -1
    p.go_for_a_walk()
    assert p.hunger == 0
```

Test-driven development

✍ Design-6: Experience test-driven development

Write a test before writing the function! You can decide yourself what your unwritten function should do, but as a suggestion it can be based on FizzBuzz - i.e. a function that:

- takes an integer argument
- for arguments that are multiples of three, returns “Fizz”
- for arguments that are multiples of five, returns “Buzz”
- for arguments that are multiples of both three and five, returns “FizzBuzz”
- fails in case of non-integer arguments or integer arguments 0 or negative
- otherwise returns the integer itself

✔ Solution

Python C++ R Julia Fortran

```python
import pytest
```

✔ Solution

- otherwise returns the integer itself

W ne tests, consider the different ways that the function could and should fa

Python C++ R Julia Fortran

A s.

```python
import pytest

def fizzbuzz(number):
    if not isinstance(number, int):
        raise TypeError
    if number < 1:
        raise ValueError
    elif number % 15 == 0:
        return "FizzBuzz"
    elif number % 3 == 0:
        return "Fizz"
    elif number % 5 == 0:
        return "Buzz"
    else:
        return number

def test_fizzbuzz():
    expected_result = [1, 2, "Fizz", 4, "Buzz", "Fizz",
                       7, 8, "Fizz", "Buzz", 11, "Fizz",
                       13, 14, "FizzBuzz", 16, 17, "Fizz", 19, "Buzz"]
    obtained_result = [fizzbuzz(i) for i in range(1, 21)]

    assert obtained_result == expected_result

    with pytest.raises(ValueError):
        fizzbuzz(-5)
    with pytest.raises(ValueError):
        fizzbuzz(0)

    with pytest.raises(TypeError):
        fizzbuzz(1.5)
    with pytest.raises(TypeError):
        fizzbuzz("rabbit")

def main():
    for i in range(1, 100):
        print(fizzbuzz(i))

if __name__ == "__main__":
    main()
```

Testing randomness

How would you test functions which generate random numbers according to some distribution/statistics?

Functions and modules which contain randomness are more difficult to test than pure deterministic functions, but many strategies exist:

- For unit tests we can use fixed random seeds.
- Try to test whether your results follow the expected distribution/statistics.
- When you verify your code "by eye", what are you looking at? Now try to express that in a script.

Consider the code below which simulates playing Yahtzee by using random numbers. How would you go about testing it?

Try to write two types of tests:

✍ a *unit test* for the `roll_dice` function. Since it uses random numbers, you will need to **set the random seed**, pre-calculate what sequence of dice throws you get with that

Consider the code below which simulates playing Yahtzee by using random numbers. How would you go about testing it?

- For unit tests we can use fixed random seeds.
- Try to test whether your results follow the expected distribution/statistics.
- When you verify your code "by eye", what are you looking at? Now try to express that in a script.

Try to write two types of tests:

✍️ a *unit test* for the `roll_dice` function. Since it uses random numbers, you will need to **set the random seed**, pre-calculate what sequence of dice throws you get with that seed, and use that in your test.

- a test of the `yahtzee` function which considers the statistical probability of obtaining a "Yahtzee" (5 dice with the same value after three throws), which is around 4.6%. This test will be an *integration test* since it tests multiple functions including the random number generator itself.

Python | C++ | R | Julia

```python
import random
from collections import Counter


def roll_dice(num_dice):
    return [random.choice([1, 2, 3, 4, 5, 6]) for _ in range(num_dice)]


def yahtzee():
    """
    Play yahtzee with 5 6-sided dice and 3 throws.
    Collect as many of the same dice side as possible.
    Returns the number of same sides.
    """

    # first throw
    result = roll_dice(5)
    most_common_side, how_often = Counter(result).most_common(1)[0]

    # we keep the most common side
    target_side = most_common_side
    num_same_sides = how_often
    if num_same_sides == 5:
        return 5

    # second and third throw
    for _ in [2, 3]:
        throw = roll_dice(5 - num_same_sides)
        num_same_sides += Counter(throw)[target_side]
        if num_same_sides == 5:
            return 5

    return num_same_sides


if __name__ == "__main__":
    num_games = 100

    winning_games = list(
        filter(
            lambda x: x == 5,
            [yahtzee() for _ in range(num_games)],
        )
    )
    print(f"Out of the {num_games} games, {len(winning_games)} got a yahtzee!")
```

Python | C++ | R | Julia

```python
def test_roll_dice():
    random.seed(0)
    assert roll_dice(5) == [4, 4, 1, 2, 5]
```

```
            [yahtzee() for _ in range(num_games)],
        )
```

Python C++ R Julia

```
'out of the {num_games} games, {len(winning_games)} got a yahtzee!")
```

```
def test_roll_dice():
    random.seed(0)
    assert roll_dice(5) == [4, 4, 1, 3, 5]
    assert roll_dice(5) == [4, 4, 3, 4, 3]
    assert roll_dice(5) == [5, 2, 5, 2, 3]


import pytest
def test_yahtzee():
    random.seed(1)
    n_tests = 1_000_000

    winning_games = list(
        filter(
            lambda x: x == 5,
            [yahtzee() for _ in range(n_tests)],
        )
    )

    assert len(winning_games) / n_tests == pytest.approx(0.046, abs=0.01)
```

Designing an end-to-end test

In this exercise we will practice designing an end-to-end test. In an end-to-end test (or integration test), the unit is the entire program. We typically feed the program with some well defined input and verify that it still produces the expected output by comparing it to some reference.

✍ Design-8: Design (but not write) an end-to-end test for the uniq program

To have a tangible example, let us consider the `uniq` command. This command can read a file or an input stream and remove consecutive repetition. The program behind `uniq` has been written by somebody else, it probably contains some functions, but we will not look into it but regard it as “black box”.

If we have a file called `repetitive-text.txt` containing:

```
(all together now) all together now
(all together now) all together now
(all together now) all together now
(all together now) all together now
(all together now) all together now
another line
another line
another line
another line
```

```
(all together now) all together now
(all together now) all together now
(all together now) all together now
(all together now) all together now
(all together now) all together now
another line
another line
another line
another line
intermission
more repetition
more repetition
more repetition
more repetition
more repetition
(all together now) all together now
(all together now) all together now
```

... then feeding this input file to `uniq` like this:

```
$ uniq < repetitive-text.txt
```

... will produce the following output with repetitions removed:

```
(all together now) all together now
another line
intermission
more repetition
(all together now) all together now
```

How would you write an end-to-end test for `uniq`?

✍ Design-9: More end-to-end testing

- Now imagine a code which reads numbers and produces some (floating point) numbers. How would you test that?
- How would you test a code end-to-end which produces images?

✍ Design-10: Create an actual end-to-end test

Often, you can include tests that run your whole workflow or program. For example, you might include sample data and check the output against what you expect. (including sample data is a great idea anyway, so this helps a lot!)

We'll use the word-count example repository https://github.com/coderefinery/word-count.

As a reminder, you can run the script like this to get some output, which prints to standard output (the terminal):

```
$ python3 code/count.py data/abyss.txt
```

Your goal is to make a test that can run this and let you know if it's successful or not. You could use Python, or you could use shell scripting. You can test if these two lines are in the output: `the 4044` and `and 2807`

Python hint: subprocess.check_output will run a command and return its output as a

count.

As a reminder, you can run the script like this to get some output, which prints to standard output (the terminal):

```
$ python3 code/count.py data/abyss.txt
```

Python hint: subprocess.check_output will run a command and return its output as a string.

Bash hint: `COMMAND | grep "PATTERN"` ("pipe to grep") will be true if the pattern is in the command.

✔ Solution

There are two solutions in the repository already, in the `tests/` dierectory https://github.com/coderefinery/word-count, one in Python and one in bash shell. Neither of these are a very advanced or perfect solution, and you could integrate them with pytest or whatever other test framework you use.

The shell one works with shell and prints a bit more output:

```
$ sh tests/end-to-end-shell.sh
the 4044
Success: 'the' found correct number of times
and 2807
Success: 'and' found correct number of times
Success
```

The Python one:

```
$ python3 tests/end-to-end-python.py
Success
```

Keypoints

- Pure functions (these are functions without side-effects) are easiest to test. And also easiest to reuse in another code project since they don't depend on any side-effects.
- Classes can be tested but it's somewhat more elaborate.

Conclusions and recommendations

The basics

- Learn one test framework well enough for basics
  - Explore and use the good tools that exist out there
  - An incomplete list of testing frameworks can be found in the Quick Reference
- Strike a healthy balance between unit tests and integration tests
- Start with some basics
- As the code gets larger and the chance of undetected bugs increases, tests should increase
  - Some simple thing that test all parts
- Automate tests
- When adding new functionality, also add tests
  - Faster feedback and reduce the number of surprises
- When you discover and fix a bug, also commit a test against this bug

Going more in-depth

- Learn one test framework well enough for basics
  - Explore and use the good tools that exist out there
  - An incomplete list of testing frameworks can be found in the Quick Reference

Going more in-depth

- Strike a healthy balance between unit tests and integration tests
- Start with some basics
- As the code gets larger and the chance of undetected bugs increases, tests should increase
  - Some simple thing that test all parts
- Automate tests
- When adding new functionality, also add tests
  - Faster feedback and reduce the number of surprises
- When you discover and fix a bug, also commit a test against this bug
- Use code coverage analysis to identify untested or unused code
- If you make your code easier to test, it becomes more modular

Ways to get started

You probably won't do everything perfectly when you start off... But what are some of the easy starting points?

- Do you have some single functions that are easy to test, but hard to verify just by looking at them? Add unit tests.
- Do you have data analysis or simulation of some sort? Make an end-to-end test with sample data, or sample parameters. This is useful as an example anyway.
- A local testing framework + GitHub actions is very easy! And works well in the background - you do whatever you want and get an email if you break things. It's actually pretty freeing.

Part 2: Automation on Forges and CI

Platforms like GitHub and GitLab offer services that can be used to automate parts of software development and maintenance.

We will see how to

- trigger remote pipelines
- run automatically the test suite of our code
- deploy documentation to GitHub Pages and GitLab Pages.

We will also discuss other possibilities these automation platform offers, and some additional, useful features.

⚙ Prerequisites

You need an account on the forge you intend to use (GitHub.com, or a GitLab server of your choice).

Automation on Forges and CI

GitHub Actions and GitLab CI/CD can automatically perform arbitrary operations, typically when someone `push`es to a repository.

Which typically means:

These are used for:

- Build and publish your work on the web (including documentation)
- Run *all* unit and integration tests, and static analysis tools
- gatekeeping/enforcing coding standard on contributions from others
- Measure test coverage
- Check code style and practices (**Linting**, auto-formatting)
- parts of the development cycle that do not fit on the machine you use to develop code.

when someone `push`es to a repository.
Which typically means:

These are used for:

- Build and publish your work on the web (including documentation)
- Run *all* unit and integration tests, and static analysis tools
- gatekeeping/enforcing coding standard on contributions from others
- Measure test coverage
- parts of the development cycle that do not fit on the machine you use to develop code.
- Check code style and practices (Linting, auto-formatting)

💬 Feedback loops in Software Development

Consider these activities we can perform on a software project:

- type checks
- incremental compilation
- full compilation
- test suite (unit)
- test suite (end-to-end)
- linting
- code format check
- building documentation
- spell checking

For each of these activities:

- How much information can it give us while developing software?
- How "expensive" is it?
- How important is it?
- Should it be done on a software forge, or on the machine where you develop?
- How often should it be performed?

What about test coverage?

✔ Writer's recommendation (but let us discuss first)

Urgency and Importance of Practices in Software Development

Practice	Information Gain	Time Cost	Importance	Where and When?
type checks	medium	low	low	local, remote (commit hooks?)
incremental compilation	medium	low	low	local
full compilation	medium	high	high	remote (local)
test suite (unit)	medium	high	very high	local,remote
test suite (end-to-end)	high	high	very high	remote (merge?) local (sparsely)
linting	medium	low	medium	local,remote (commit hooks)
building documentation	low	medium	high	remote
spell check	low	low	medium	remote (local)

Typically test coverage measurement and display can be performed remotely.
Coverage data is gathered when running a test suite.

Git Hooks

Git has a mechanism to trigger actions when some event happen, called *hooks*.

linting	medium	low	medium	local,remote (commit hooks)
building documentation	low	medium	high	remote
spellcheck	low	low	medium	remote (local)

Typically test coverage measurement and display can be performed remotely.
Coverage data is gathered when running a test suite.

Git Hooks

Git has a mechanism to trigger actions when some event happen, called *hooks*.

Most notable hooks:

- `pre-commit`: launches a process when you launch the command `git commit`. If the process terminates with a non-success exit code, it will prevent you from committing your changes. Typical use cases:
  - auto formatting and formatting checks
  - linting

  Note

  There is a Python framework called pre-commit created to manage (e.g., install and run) useful pre-commit hooks.
- `post-receive`: this hook can be used to launch a process on the machine where the remote repository is hosted, as a fundamental form of CI.

✍ Basic CX with hooks

We can reproduce the fundamental functionality of automation platforms by setting up a bare repository on a remote machine and the relevant hooks.

For this exercise:

1. clone this repository:

```
git clone https://gitlab.com/michele.mesiti/cx-course.git
```

2. switch to the `hooks` branch
3. follow the instructions in the README.

Start with the “local” version of the exercise, then - if you have a remote machine you can connect to via SSH - try the “remote” version.

Note that this simple mechanism is very limited, which is why in most cases automation platforms like GitHub actions or Gitlab CI/CD are used.

Testing on software forges

- Test and **benchmark the performance** on specific hardware and infrastructure
- build/package software for various compilers, MPI versions, architectures
- … all using standardized environments in conjunction with containers

Some issues in the development of HPC code are addressed by regularly running jobs that:
This might require “special” set up, especially performance benchmarking.

- Test compilation with different compilers and libraries (e.g., MPIs, CUDA/ROCm, …)

- Test and **benchmark the performance** on specific hardware and infrastructure
- build/package software for various compilers, MPI versions, architectures
- ... all using standardized environments in conjunction with containers

Some issues in the development of HPC code are addressed by regularly running jobs that:

This might require "special" set up, especially performance benchmarking.

- Test compilation with different compilers and libraries (e.g., MPIs, CUDA/ROCm, ...)

Testing on software forges

❓ Questions

- How can we implement automatic testing each time we push changes to the repository?
- Why is it good to autoclose issues with commit messages?

"Continuous integration"

We will now learn to set up automatic tests using either GitHub Actions or GitLab CI/CD - you can choose which one to use and instructions are provided for both.

This exercise can be run in "collaborative mode" by following instead the instructions in Full collaborative workflow. In the collaborative version steps C-D below are performed by a collaborator.

✍ Exercise CI-1: Create and use a continuous integration workflow on GitHub or GitLab

In this exercise, we will:

- **A.** Create and add code to a repository on GitHub/GitLab (or, alternatively, fork and clone an existing example repository)
- **B.** Set up tests with GitHub Actions/ GitLab CI/CD
- **C.** Find a bug in our repository and open an issue to report it
- **D.** Fix the bug on a bugfix branch and open a pull request (GitHub)/ merge request (GitLab)
- **E.** Merge the pull/merge request and see how the issue is automatically closed.
- **F.** Create a test to increase the code coverage of our tests.

Prerequisites

If you are new to Git, you can find a step-by-step guide to setting up repositories and making commits in this git-refresher material. If you are new to pull requests / merge requests, you can learn all about them in the Collaborative Git lesson.

Step 1: Create a new repository on GitHub/GitLab OR fork from the example repo

- Add the following files and code

Create a new repository

Python

reating a repository called (for example) *example-ci*.
u create the repository, select **"Initialize this repository with a README"**

Add a file `functions.py` containing:

- Add the following files and code

Create a new repository

Python R

reating a repository called (for example) *example-ci*.
u create the repository, select **"Initialize this repository with a README"**

Add a file `functions.py` containing:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a + b  # <--- fix this in step 7

def multiply(a, b):
    return a * b

def convert_fahrenheit_to_celsius(fahrenheit):
    return multiply(subtract(fahrenheit, 32), 9 / 5) # <-- Fix this in step 7
```

and a file `test_functions.py` containing:

```
from functions import add, subtract, multiply
from functions import convert_fahrenheit_to_celsius as f2c
import pytest

def test_add():
    assert add(2, 3) == 5
    assert add('space', 'ship') == 'spaceship'

# uncomment the following test in step 5
#def test_subtract():
#    assert subtract(2, 3) == -1

# uncomment the following test in step 11
# def test_convert_fahrenheit_to_celsius():
#    assert f2c(32) == 0
#    assert f2c(122) == pytest.approx(50)
#    with pytest.raises(AssertionError):
#        f2c(-600)
```

Finally, stage the files (`git add <filename>`), commit (`git commit -m "some commit message"`), and push the changes (`git push origin main`).

Fork and clone an existing example repository

- Fork the example repo. There are two options one for Python and one for R.
- Clone your fork (`git clone git@github.com:<yourGitID>/<Py/R>TestingExample.git`).

Step 2: Run tests locally

```
pytest
```

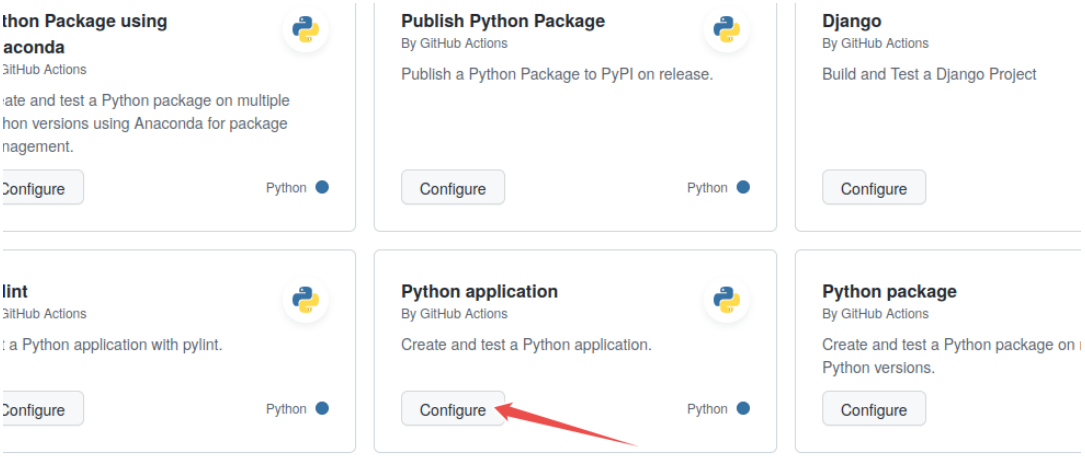
You can now run your tests locally with

```
pytest
```

You can now run your tests locally with

GitHub-Python | GitLab-Python | GitHub-R

In this step we will enable GitHub Actions. Select “Actions” from your GitHub repository page. You get to a page “Get started with GitHub Actions”. Select the button for “Configure” under Python Application:



Select “Python application” as the starter workflow.

GitHub creates the following file for you in the subfolder `.github/workflows`. Modify the highlighted lines according to the action below. This will add a code coverage report to new pull requests. The if clause restricts this to pull requests, as otherwise this action would not have a target to write the reports to. On pushes only the unittesting is run.

```
# This workflow will install Python dependencies, run tests and lint with a single
version of Python
# For more information see: https://docs.github.com/en/actions/automating-builds-
and-tests/building-and-testing-python

name: Test

on:
  push:
```

```yaml
# This workflow will install Python dependencies, run tests and lint with a single version of Python
# For more information see: https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python

name: Test

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

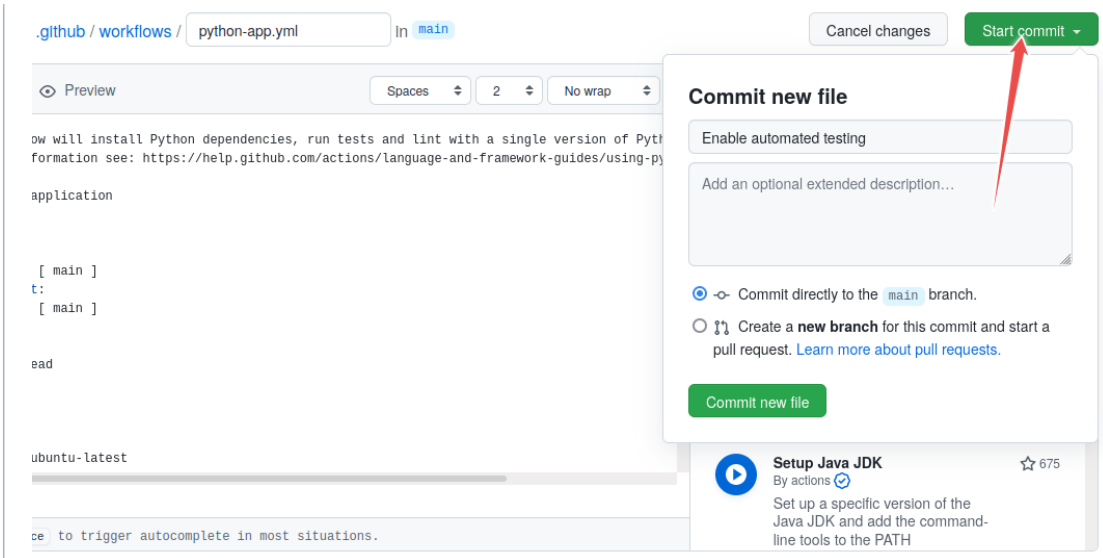
permissions:
  contents: read
  pull-requests: write

jobs:
  build:

    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.10
      uses: actions/setup-python@v3
      with:
        python-version: "3.10"
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install flake8 pytest pytest-cov
        if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
    - name: Lint with flake8
      run: |
        # stop the build if there are Python syntax errors or undefined names
        flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
        # exit-zero treats all errors as warnings. The GitHub editor is 127 chars wide
        flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --statistics
    - name: Test with pytest
      run: |
        pytest --cov-report "xml:coverage.xml" --cov=.
    - name: Create Coverage
      if: ${{ github.event_name == 'pull_request' }}
      uses: orgoro/coverage@v3
      with:
          coverageFile: coverage.xml
          token: ${{ secrets.GITHUB_TOKEN }}
```

Commit the change by pressing the “Start Commit” button:

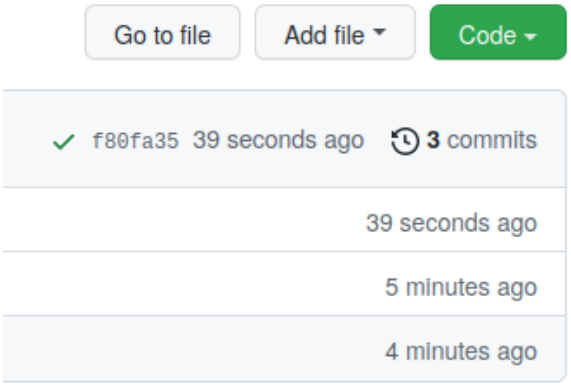


Committing the file via the GitHub web interface: follow the flow, give it some commit name. You can commit directly to master.

Step 4: Verify that tests have been automatically run

GitHub-Python | GitLab-Python | GitHub-R

Observe in the repository how the test succeeds. While the test is executing, the repository has a yellow marker. This is replaced with a green check mark, once the test succeeds:



Green check means passed.

Also browse the “Actions” tab and look at the steps there and their output.

Step 5: Add a test which reveals a problem

After you committed the workflow file, your GitHub/GitLab repository will be ahead of your local cloned repository. Update your local cloned repository:

```
$ git pull origin main
```

Next uncomment the code in `test_functions.py` under “step 5”, commit, and push. Verify

Step 6: Open an issue on GitHub/GitLab

Open a new issue in your repository about the broken test (click the “Issues” button on GitHub or GitLab and write a title for the issue). The plan is that we will fix the issue through

Hint: if the above command fails, check whether the branch name on the GitHub/GitLab repository is called `main` and not perhaps `master`.

Next uncomment the code in `test_functions.py` under “step 5”, commit, and push. Verify

```
$ git pull origin main
```

Step 6: Open an issue on GitHub/GitLab

Open a new issue in your repository about the broken test (click the “Issues” button on GitHub or GitLab and write a title for the issue). The plan is that we will fix the issue through a pull/merge request.

Step 7: Fix the broken test

Now fix the code **on a new branch**, you can call it `yourname/bugfix`. After you have fixed the code on the new branch, commit the following commit message `"restore function subtract; fixes #1"` (assuming that you try to fix issue number 1).

Shortcut

Here it’s perfectly possible to take a shortcut and commit and push directly to the main branch. If you do this, steps 8-9 below are skipped.

- When would you push directly to the main branch, and when would you send a pull/merge request?

Then push to your repository.

Step 8: Open a pull request (GitHub)/ merge request (GitLab)

Go back to the repository on GitHub or GitLab and open a pull/merge request. **In a collaborative setting, you could request a code review from collaborators at this stage.** Before accepting the pull/merge request, observe how GitHub Actions/ Gitlab CI automatically tested the code.

If you forgot to reference the issue number in the commit message, you can still add it to the pull/merge request: `my pull/merge request title, closes #1`.

Step 9: Accept the pull/merge request

Observe how accepting the pull/merge request automatically closes the issue (provided the commit message or the pull/merge request contained the correct issue number).

See also:

- GitHub: closing issues using keywords
- GitLab: closing issues using keywords

Step 10: Increase your code coverage

We are currently missing several functions in our tests. Write a test for the `multiply` function in a new branch and create a pull request. On Python you can directly observe the increase in code coverage. On R you can have a look at the action (`Actions -> last run of your action -> Select a job -> Test coverage`). If you compare this with the previous run, you should see an increase once the update is in.

Step 11 (optional): Repeat steps 5-9 for the `convert_fahrenheit_to_celsius` function:

Discuss whether this is a useful feature. And if it is, why do you think is it u

We are currently missing several functions in our tests. Write a test for the `multiply` function in a new branch and create a pull request. On Python you can directly observe the increase in code coverage. On R you can have a look at the action (`Actions -> last run of your action -> Select a job -> Test coverage`). If you compare this with the previous run, you should see an increase once the update is in.

Step 11 (optional): Repeat steps 5-9 for the `convert_fahrenheit_to_celsius` function:

Repetition helps learning, so let's do the testing again for our `convert_fahrenheit_to_celsius` function. Uncomment the test for the `convert_fahrenheit_to_celsius` function and repeat steps 5 to 9 fixing the bug this test exposes.

Discussion

Finally, we discuss together about our experiences with this exercise.

Where to go from here

- This example was using Python but you can achieve the same automation for R or Fortran or C/C++ or other languages
- This workflow is very useful for collaborators who work on the same code and it works both for centralized and forking workflows - have a look at this alternative exercise to see how that works.
- GitHub Actions has a Marketplace which offer wide range of automatic workflows
- On GitLab use GitLab CI
- For Windows builds you can also use Appveyor

Keypoints

- When fixing bugs or other problems reported in issues, use the issue autoclosing mechanism when you send the pull/merge request.

Testing on software forges: Full collaborative workflow

Questions

- How can we implement automatic testing each time we push changes to the repository?
- Why is it good to autoclose issues with commit messages?

Exercise a full-cycle collaborative workflow

This exercise is a collaborative version of the Automated testing exercise.

In this exercise, everybody will:

A. Create a repository on GitHub/GitLab (everybody should use a **different repository name** for their repository)
B. Commit code to the repository and set up tests with GitHub Actions/ GitLab CI
C. Everybody will find a bug in their repository and open an issue in their repository
D. Then each one will clone the repo of one of their exercise partners, fix the bug, and

✍ Set up a continuous integration workflow on GitHub or GitLab with pull requests and issues

This is an expanded version of the automated testing demonstration. The exercise is performed in a collaborative circle within the exercise group (breakout room).
The exercise takes 20-30 minutes.

In this exercise, everybody will:

✍️ Practice: Create and use a continuous integration workflow on GitHub or GitLab with pull requests and issues

This is an expanded version of the automated testing demonstration. The exercise is performed in a collaborative circle within the exercise group (breakout room).

The exercise takes 20-30 minutes.

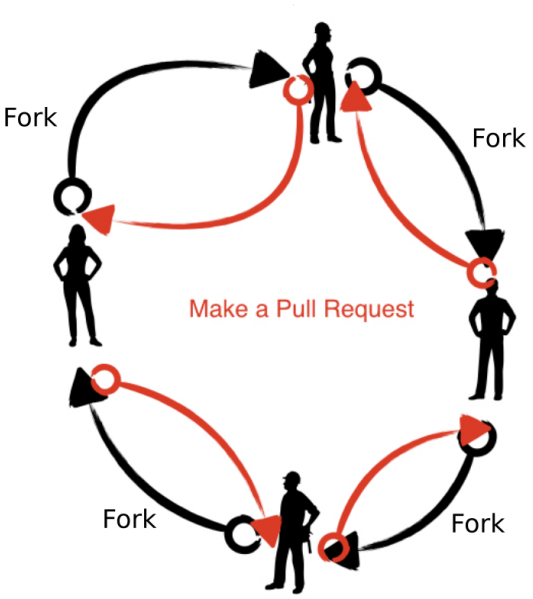
A. Create a repository on GitHub/GitLab (everybody should use a **different repository name** for their repository)

B. Commit code to the repository and set up tests with GitHub Actions/ GitLab CI

C. Everybody will find a bug in their repository and open an issue in their repository

D. Then each one will clone the repo of one of their exercise partners, fix the bug, and open a pull request (GitHub)/ merge request (GitLab)

E. Everybody then merges their co-worker's change



Overview of this exercise. Below we detail the steps.

Prerequisites

If you are new to Git, you can find a step-by-step guide to setting up repositories and making commits in this git-refresher material.

Step 1: Create a new repository on GitHub/GitLab

- Select a **different repository name** than your colleagues (otherwise forking the same name will be strange)
- **Before** you create the repository, select **"Initialize this repository with a README"** (otherwise you try to clone an empty repo).
- Share the repository URL with your exercise group via shared document or chat

Step 2: Clone your own repository, add code, commit, and push

Clone the repository.

Add a file `example.py` containing:

```
def add(a, b):
    return a + b


def test_add():
    assert add(2, 3) == 5
    assert add('space', 'ship') == 'spaceship'
```

```
def add(a, b):
    return a + b


def test_add():
    assert add(2, 3) == 5
    assert add('space', 'ship') == 'spaceship'


def subtract(a, b):
    return a + b  # <--- fix this in step 8


# uncomment the following test in step 5
#def test_subtract():
#    assert subtract(2, 3) == -1
```

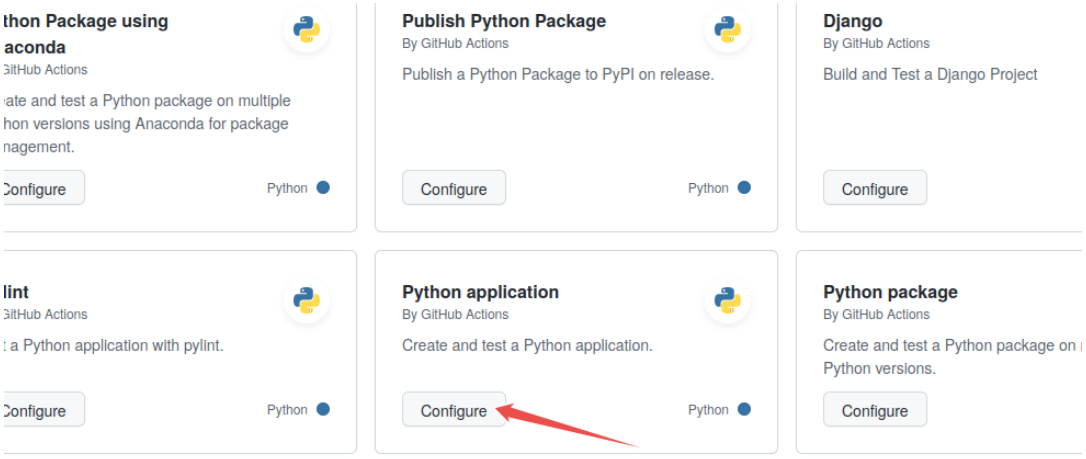
Test `example.py` with `pytest`.

Then stage the file (`git add <filename>`), commit (`git commit -m "some commit message"`), and push the changes (`git push origin main`).

Step 3: Enable automated testing

GitHub | GitLab

In this step we will enable GitHub Actions. Select “Actions” from your GitHub repository page. You get to a page “Get started with GitHub Actions”. Select the button for “Set up this workflow” under Python Application:



Select “Python application” as the starter workflow.

GitHub creates the following file for you in the subfolder `.github/workflows`. Add `pytest example.py` to the last line (highlighted):

```
# This workflow will install Python dependencies, run tests and lint with a single
version of Python
# For more information see: https://help.github.com/actions/language-and-
framework-guides/using-python-with-github-actions

name: Python application

on:
  push:
```

```yaml
# This workflow will install Python dependencies, run tests and lint with a single
version of Python
# For more information see: https://help.github.com/actions/language-and-
framework-guides/using-python-with-github-actions

name: Python application

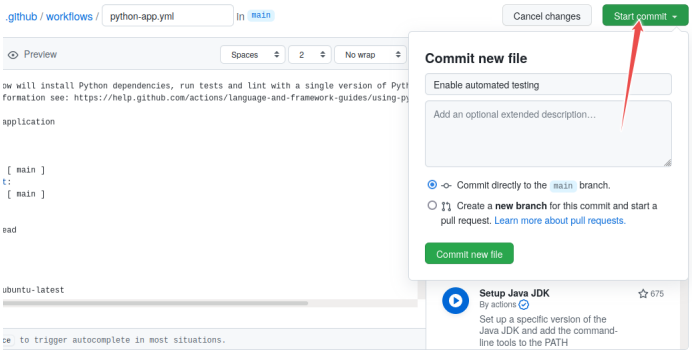
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:

    runs-on: ubuntu-latest

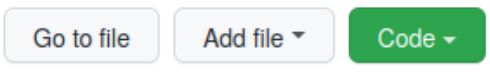
    steps:
    - uses: actions/checkout@v2
    - name: Set up Python 3.8
      uses: actions/setup-python@v2
      with:
        python-version: 3.8
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install flake8 pytest
        if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
    - name: Lint with flake8
      run: |
        # stop the build if there are Python syntax errors or undefined names
        flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
        # exit-zero treats all errors as warnings. The GitHub editor is 127 chars
wide
        flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --
statistics
    - name: Test with pytest
      run: |
        pytest example.py
```

Commit the change by pressing the “Start Commit” button:

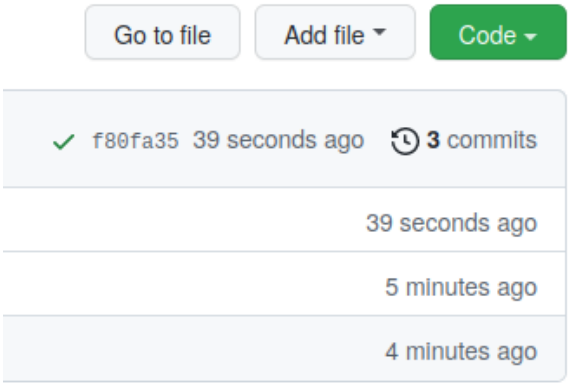


Committing the file via the GitHub web interface: follow the flow, give it some commit name. You can commit directly to master.

Observe in the repository how the test succeeds. While the test is executing, the repository has a yellow marker. This is replaced with a green check mark, once the test succeeds:



Observe in the repository how the test succeeds. While the test is executing, the repository has a yellow marker. This is replaced with a green check mark, once the test succeeds:



Green check means passed.

Also browse the “Actions” tab and look at the steps there and their output.

Step 5: Add a test which reveals a problem

After you committed the workflow file, your GitHub/GitLab repository will be ahead of your local cloned repository. Update your local cloned repository:

```
$ git pull origin main
```

Hint for helpers: if the above command fails, check whether the branch name on the GitHub/GitLab repository is called `main` and not perhaps `master`.

Next uncomment the code in `example.py` under “step 5”, commit, and push. Verify that the test suite now fails on the “Actions” tab (GitHub) or the “CI/CD->Pipelines” tab (GitLab).

Step 6: Open an issue on GitHub/GitLab

Open a new issue in your repository about the broken test (click the “Issues” button on GitHub or GitLab and write a title for the issue). The plan is that your colleague will fix the issue through a pull/merge request.

Step 7: Fork and clone the repository of your colleague

Fork the repository using the GitHub/GitLab web interface. (if you are unfamiliar with forking and pull requests, have a look at this visual representation).

```
$ git clone https://github.com/your-username/the-repository.git
```

Step 8: Fix the broken test

```
$ git clone https://github.com/your-username/the-repository.git
```

Step 8: Fix the broken test

After you have fixed the code, commit the following commit message `"restore function subtract; fixes #1"` (assuming that you try to fix issue number 1).

Then push to your fork.

Step 9: Open a pull request (GitHub)/ merge request (GitLab)

Then before accepting the pull request/ merge request from your colleague, observe how GitHub Actions/ Gitlab CI automatically tested the code.

If you forgot to reference the issue number in the commit message, you can still add it to the pull request/ merge request: `my pull/merge request title, closes #NUMBEROFTHEISSUE`

Step 10: Accept the pull request/ merge request

Observe how accepting the pull request/ merge request automatically closes the issue (provided the commit message or the pull request/ merge request contained the correct issue number).

See also:

- GitHub: closing issues using keywords
- GitLab: closing issues using keywords

Discuss whether this is a useful feature. And if it is, why do you think is it useful?

Discussion

Finally, we discuss together about our experiences with this exercise.

✍ (optional) FullCI-2: Add a license file to the previous exercise's repository

In the Social coding and open software lesson we learn how important it is to add a LICENSE file.

Your goal:

- You discover that your coworker's repository does not have a LICENSE file.
- Open an issue and suggest a LICENSE.
- Then add a LICENSE via a pull/merge request, referencing the issue number.

Where to go from here

- This example was using Python but you can achieve the same automation for R or Fortran or C/C++ or other languages
- GitHub Actions has a Marketpace which offer wide range of automatic workflows
- On GitLab use GitLab CI
- For Windows builds you can also use Appveyor

- Open an issue and suggest a LICENSE.
- Then add a LICENSE via a pull/merge request, referencing the issue number.

Where to go from here

- This example was using Python but you can achieve the same automation for R or Fortran or C/C++ or other languages
- GitHub Actions has a Marketpace which offer wide range of automatic workflows
- On GitLab use GitLab CI
- For Windows builds you can also use Appveyor

Keypoints

- When fixing bugs or other problems reported in issues, use the issue autoclosing mechanism when you send the pull/merge request.

Deploying Sphinx documentation

Objectives

- Create a basic workflow for deploying documentation which you can take home and adapt for your project.

GitHub Pages and GitLab Pages serve websites from a GitHub/GitLab repository.

We will let them run `sphinx-build` and make the result available on GitHub/GitLab Pages.

We host source code with documentation sources on a public Git repository, and we want to build the documentation as a static website and serve it.

Each time we `git push` to the repository, a GitHub action or a GitLab pipeline triggers to rebuild the documentation.

Demonstration - Deploy Sphinx documentation to GitHub/GitLab Pages

✍ GH-Pages-1: Deploy Sphinx documentation to GitHub or GitLab Pages

In this exercise we will create an example repository on a software forge (GitHub or GitLab) and deploy it to the corresponding "Page" service (GitHub pages or GitLab pages)

Preliminary: Verify the "Pages" feature is available on your forge

GitHub | GitLab

On `github.com`, the feature is typically available.

GitHub | GitLab

On GitHub the most convenient way is to generate the repository from a template.

Go to the documentation-example project template on GitHub and create a copy to your namespace.

GitHub | GitLab

On GitHub the most convenient way is to generate the repository from a template.

Go to the documentation-example project template on GitHub and create a copy to your namespace.

- Give it a name, for instance “documentation-example”.
- You don’t need to “Include all branches”
- Click on “Create a repository”.

Step 2: Browse the new repository.

- The documentation part of the project is in `doc/` (many projects do it this way).
- The source code for your project could then go under `src/`.

Step 3: Automate the generation of the documentation

GitHub | GitLab

Add the GitHub Action to your new Git repository.

- Add a new file at `.github/workflows/documentation.yml` (either through terminal or web interface), containing:

```yaml
name: documentation

on: [push, pull_request, workflow_dispatch]

permissions:
  contents: write

jobs:
  docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - name: Install dependencies
        run: |
          pip install sphinx sphinx_rtd_theme myst_parser
      - name: Sphinx build
        run: |
          sphinx-build doc _build
      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        if: ${{ github.event_name == 'push' && github.ref ==
'refs/heads/main' }}
        with:
          publish_branch: gh-pages
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: _build/
          force_orphan: true
```

previous episode).

- After the file has been committed (and pushed), check the action at `https://github.com/USER/documentation-example/actions` (replace `USER` with your

```
        with:
          publish_branch: gh-pages
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: _build/
          force_orphan: true
```

previous episode).

- After the file has been committed (and pushed), check the action at `https://github.com/USER/documentation-example/actions` (replace `USER` with your GitHub username).

Step 4: Enable Pages feature and verify it's running

GitHub | GitLab

Note that once the documentation created it is pushed to a separate branch called 'gh-pages' (this is what the action `peaceiris/actions-gh-pages` does).

- Go to "Settings" -> "Pages".
- Under "Build and deployment"
  - In the **Source** section: choose "Deploy from a branch" in the dropdown menu
  - In the **Branch** section: choose "gh-pages" and "/ (root)" in the dropdown menus and click the **Save** button.
- You should now be able to verify the pages deployment in the "Actions" list (this is how it looks like for this lesson material).
- You can verify that the repository has now a branch named `gh-pages` which contains the generated documentation (and only that).

Step 5: Verify the result

GitHub | GitLab

Your site should now be live at `https://USER.github.io/documentation-example/` (replace USER).

Step 6 (optional): Verify refreshing the documentation

- Commit some changes to your documentation
- Verify that the documentation website refreshes after your changes (can take few seconds or a minute)

Exercise - Sphinx documentation on GitHub Pages

✍️ GH together

1. Follow the above instructions to create a new repository with a Sphinx documentation project;
2. Try adding one or more of the following to your Sphinx project:
   1. API documentation (see exercise in part 1 on API references) which requires the `sphinx-autodoc2` package.
   2. a Jupyter notebook (see exercise in part 1 on Jupyter notebooks) which requires the `myst-nb` package.
   3. change the theme (see the end of the quickstart in part 1). You can browse themes and find their package names on the Sphinx themes gallery.

1. API documentation (see exercise in part 1 on API references) which requires the `sphinx-autodoc2` package.
2. a Jupyter notebook (see exercise in part 1 on Jupyter notebooks) which requires the `myst-nb` package.
3. change the theme (see the end of the quickstart in part 1). You can browse themes and find their package names on the Sphinx themes gallery.

✍ GH together

1. Follow the above instructions to create a new repository with a Sphinx documentation project;
2. Try adding one or more of the following to your Sphinx project:

❗ Important

The computer on which the GitHub actions run is *not* your local machine, and might not have the libraries you need to build the project. Make sure you update the dependencies (installed with `pip` in the demonstration) appropriately.

❗ Important

Make sure the correct file paths are used. This will require adjusting paths from the example from the previous episode to the new layout. Note many paths, including e.g. the `autodoc2_packages` preference are now relative to the `doc/` directory.

What do you need to change in the workflow file?

✔ Solution

1. **API documentation**
    1. In the workflow/pipeline definition file (`.github/workflows/documentation.yml` or `.gitlab-ci.yml`), when calling `pip`, add the package `sphinx-autodoc2` to the list of packages to install.
    2. Follow the instructions in Sphinx-3 changing paths so that:
        1. `multiply.py` is `src/multiply.py` and is specified as `../src/multiply.py` in the `autodoc2_packages` preference in `conf.py`
        2. `conf.py` is `doc/conf.py`
        3. `index.md` is `doc/index.md`.
    3. Commit and push your changes, verify the action has run successfully, and view the built site in your browser.
2. **a Jupyter notebook**
    1. In the workflow/pipeline definition file (`.github/workflows/documentation.yml` or `.gitlab-ci.yml`), when calling `pip`, add the package `myst-nb` to the list of packages to install.
    2. Follow the instructions in Sphinx-4 changing paths so that:
        1. `flower.md` is `doc/flower.md`
        2. `conf.py` is `doc/conf.py`
        3. `index.md` is `doc/index.md`.
    3. Commit and push your changes, verify the action has run successfully, and view the built site in your browser.
3. **change the theme**
    1. In the workflow/pipeline definition file (`.github/workflows/documentation.yml` or `.gitlab-ci.yml`), when calling `pip`, and add the theme package if necessary.
    2. In `doc/conf.py` change `html_theme = 'sphinx_rtd_theme'` to the name of your chosen theme.
    3. Commit and push your changes, verify the action has run successfully, and view the built site in your browser.

Alternatives to GitHub and GitLab pages

- Read the Docs is the most common alternative to hosting in GitHub Pages.

3. Change the theme
    3. Commit and push your changes, verify the action has run successfully, and view the built site in your browser.
    1. In the workflow/pipeline definition file (`.github/workflows/documentation.yml` or `.gitlab-ci.yml`), when calling `pip`, and add the theme package if necessary.
    2. In `doc/conf.py` change `html_theme = 'sphinx_rtd_theme'` to the name of your chosen theme.

Alternatives to GitHub and GitLab pages

- Read the Docs is the most common alternative to hosting in GitHub Pages.
- Sphinx builds HTML files (this is what static site generators do), and you can host them anywhere, for example your university's web space or own web server.

Migrating your own documentation to Sphinx

- First convert your documentation to Markdown using Pandoc.
- Create a file `index.rst` which lists all other Markdown files and provides the table of contents.
- Add a `conf.py` file. You can generate a starting point for `conf.py` and `index.rst` with `sphinx-quickstart`, or you can take the examples in this lesson as inspiration.
- Test building the documentation locally with `sphinx-build`.
- Once this works, follow the above steps to build and deploy to GitHub Pages or some other web space.

Keypoints

- Sphinx makes simple HTML (and more) files, so it is easy to find a place to host them.
- Github Pages + Github Actions provides a convenient way to make sites and host them on the web.

Hosting websites/homepages on GitHub Pages

Often we don't need more than a static website

You can host your personal homepage or group webpage or project website on GitHub using GitHub Pages. GitLab and Bitbucket also offer a very similar solution.

Unless you need user authentication or a sophisticated database behind your website, GitHub Pages can be a very nice alternative to running your own web servers. This is how all https://coderefinery.org material is hosted.

How to find the website URL

Here below, NAMESPACE can either be a username or an organizational account.

Personal homepage or organizational homepage

- Generated URL: https://**NAMESPACE**.github.io
- Generated from: https://github.com/**NAMESPACE**/**NAMESPACE**.github.io (main branch)

Project website

- Generated URL: https://**NAMESPACE**.github.io/**REPOSITORY**
- Generated from: https://github.com/**NAMESPACE**/**REPOSITORY** (gh-pages branch)

Exercise - Your own website on GitHub Pages

GH-Pages-2: Host your own github page

- Deploy own website reusing a template:
    - Follow the steps from GitHub Pages https://pages.github.com/. The documentation there is very good, so there is no need for us to duplicate the

- Generated URL: https://**NAMESPACE**.github.io/
- Generated from: https://github.com/**NAMESPACE**/**NAMESPACE**.github.io (main branch)

Exercise - Your own website on GitHub Pages

✍ GH-Pages-2: Host your own github page

Project website

- Deploy own website reusing a template:
  - Follow the steps from GitHub Pages https://pages.github.com/. The documentation there is very good, so there is no need for my to duplicate the screenshots.
  - Select “Project site”.
  - Select “Choose a theme”.
  - Follow the instructions on https://pages.github.com/.
  - Browse your page on https://**USERNAME**.github.io/**REPOSITORY** (adjust “USERNAME” and “REPOSITORY”).
- Make a change to the repository after the webpage has been deployed for the first time.
- Please wait few minutes and then verify that the change shows up on the website.

- Generated URL: https://**NAMESPACE**.github.io/**REPOSITORY**
- Generated from: https://github.com/**NAMESPACE**/**REPOSITORY** (gh-pages branch)

Real-life examples

- CodeRefinery website (built using Zola)
  - Source
  - Result: https://coderefinery.org/lessons/
- This lesson (built using Sphinx and MyST and sphinx-lesson)
  - Source
  - Result: this page

Note

- You can use HTML directly or another static site generator if you prefer to not use the default Jekyll.
- It is no problem to use a custom domain instead of `*.github.io`.

Other features

Objectives

- Have an idea of what is possible to do with actions or CI, and how

In the past episodes we have seen some examples of minimal workflows (testing, documentation).

In this episode we focus on specific features. In the exercise repository (see link to GiHub version, link to GiLab version) a number of examples and exercises have been collected, so that you can try different features.

GitHub Actions GitLab CI/CD

you prefer.

The workflow are defined in `yaml` files in `.github/workflows` (one file - one workflow)

- Typically each push triggers the **workflow**s unless skipped
- Each workflow has

GitHub Actions | GitLab CI/CD

you prefer.

The workflow are defined in `yaml` files in `.github/workflows` (one file - one workflow)

- Typically each push triggers the **workflow**s unless skipped
- Each workflow has
  - a list of event triggers (typically `push`, but others are possible, for example `workflow_dispatch` which is necessary to launch the workflow manually)
  - a list of jobs, that run independently, made of steps
    - each job has a list of steps that run one after another
    - steps runs on the same "machine" and share data and context
    - each step can use a pre-defined *action* (see) or a script
    - the first step typically is the checkout action
    - steps are executed one after the other. If a step fails, the subsequent steps are not executed, and the job fails.
  - if a job in a workflow fails, then the workflow fails
- if a workflow fails, then the build is marked as failed.

Note

Editing workflow and pipeline files

Whenever you want to edit a file that defines a workflow (on GitHub) or a pipeline (on GitLab) consider doing this directly on the web interface, as the build-in editors have a linter and autocompletion which will vastly reduce the probability of making trivial mistakes.

Warning

Exercises on GitHub: starting workflows manually

To trigger manually the workflows on GitHub using your fork of example repository, you might have to switch the default branch to the appropriate one, in "Settings" -> "General" -> "Default Branch".

Note

Email notifications

Both GitHub and GitLab are eager to send you emails when a workflow or pipeline fails.

While this is in most case very useful, consider disabling email notifications on the repository you use for the exercises during this session.

GitHub Actions | GitLab CI/CD

Compile and run a C program

In this case we have a single job, with 3 steps: checkout, build and run.

```
name: BasicExample
on:
```

GitHub Actions GitLab CI/CD

Compile and run a C program

In this case we have a single job, with 3 steps: checkout, build and run.

```yaml
name: BasicExample
on:
  - push                                # the workflow runs when we push
  - workflow_dispatch                   # we can launch the workflow manually

jobs:
  build_and_run:
    runs-on: ubuntu-latest              # This is necessary,
                                        # specifies the kind of host where to
run
    steps:
      - name: "Checkout"
        uses: actions/checkout@v6       # We use a pre-defined *action*
      - name: "build"
        run: gcc -o hello ./src/hello.c
      - name: "run"
        run: ./hello
```

Note that `runs-on` takes one or more labels of a runner, and identifies the type of host.

Failures

Proper reporting and propagation of the failure of a command in a pipeline or workflow is **paramount**.

A CI job not properly reporting a failure is itself a severe bug.

Switch to the branch `failures`.

The shell script `./scripts/doesnt-fail-but-typically-should.sh` shows a typical pitfall.

- How can the problem be fixed?
- What is the default behaviour of on GitHub or GitLab, when writing the logic in a workflow/pipeline file instead of a separate shell script?

Switch to the branch `failures` and follow the instructions in the README, and have a look at the workflow/pipeline definition file.

Secrets and repository-specific behaviour

In some cases we want to change the behaviour of the workflows/pipelines depending on the repository.

Check out the branch `environment-variables` in the example repository, to see how to customize a job by using environment variables that are set at the **repository** level.

In other cases, we might want to avoid storing some information under version control (it might be not general enough, or it might be a secret). but it is needed to run workflows or pipelines.

The solution is to use environment variables (also *secrets* on GitHub).

Artifacts

Artifacts are the main way to transfer information between jobs in a GitLab pipeline, and from the runner to the GitLab web interface.

Check out the branch `environment-variables` in the example repository, to see how to customize a job by using environment variables that are set at the **repository** level.

In other cases, we might want to avoid storing some information under version control (it might be not general enough, or it might be a secret). but it is needed to run workflows or pipelines.

The solution is to use environment variables (also *secrets* on GitHub).

Artifacts

Artifacts are the main way to transfer information between jobs in a GitLab pipeline, and from the runner to the GitLab web interface.

GitHub | GitLab CI/CD

Code reuse

When creating a large workflow or pipeline, we might be tempted to copy/paste the job definition. What alternatives do we have?

GitHub Actions | GitLab CI/CD

YAML anchors also work on GitHub actions (with some limitations compared to GitLab).

In the GitHub context, actions themselves are the main building block, and they are reusable by default.

Parametric tasks are also a way to "reuse" code, or at least to avoid code duplication.

Parametric tasks: Matrix

Sometimes you might want to run the same job for many different "cases", or parameters (e.g., different versions of a library/dependency/compiler).

GitHub Actions | GitLab CI/CD

Different variations of a job in a workflow can be run by using a matrix strategy.

Mirroring

A *Mirror* is a copy of a repository that is kept *automatically* in sync while being, typically, on a different forge, to take advantages of different features, or reach different communities.

There are 2 possible techniques:

GitHub Actions | GitLab CI/CD

pository is configured to automatically pull from the original at regular

To set a push mirror, one can use GitHub actions (using the action https://github.com/wangchucheng/git-repo-sync)

A small guide is available in the `github-to-gitlab-mirror` branch of the example

GitHub Actions | GitLab CI/CD

pository is configured to automatically pull from the original at regular

To set a push mirror, one can use GitHub actions (using the action https://github.com/wangchucheng/git-repo-sync)

A small guide is available in the `github-to-gitlab-mirror` branch of the example repository.

When triggered, the workflow on that branch pushes the current branch of the repository to a mirror on GitLab.com.

✍ Push mirror from GitHub to GitLab

1. Fork the example repository on GitHub.
2. Create and empty repository on a GitLab server (tip: disable notifications)
3. Checkout the `github-to-gitlab-mirror` branch in the example repo
4. Follow the instructions in the README.md to set up the mirroring

Conditional execution of workflows, jobs and pipelines

On both platforms it is possible to make so that the execution of the jobs, steps or of the full workflow is dependent on some conditions.

Typical cases are:

- Always: force the execution of jobs/steps no matter what
- Only in case of a merge
- Only in case of a tagged commit
- Only on a particular branch
- based on expressions involving environment variables
- only when some files are changed

GitHub Actions | GitLab CI/CD

Self-hosting

There might be situations where you need to run the workflows/pipelines on resources you own, instead of relying on `github.com` or any specific gitlab server.

Possible reasons:

- GitHub actions and GitLab CI/CD on gitlab.com have usage limits (for private repos), or reliability issues, and you get builds take too long
- You need to test (or benchmark) your code with specific hardware and software (e.g., on

Both gitlab and github will schedule jobs on runners comparing the tags/labels of the runners and the tags/labels of the job. For a job to execute on a given runner, the tags/labels of the job must be a subset of the tags/labels of the runner.

GitHub Actions | GitLab CI/CD

Both gitlab and github will schedule jobs on runners comparing the tags/labels of the runners and the tags/labels of the job to execute on a given runner. The tags/labels of the job must be a subset of the tags/labels of the runner.

- GitHub Actions and GitLab CI/CD on gitlab.com have usage limits (for private repos) or reliability issues, and your test suites take too long
- You need to test (or benchmark) your code with specific hardware and software (e.g., on

GitHub Actions | GitLab CI/CD

One can add a self-hosted runner to a repository.

Warning

It is a security risk to have a self-hosted runner attached to public repositories, because in that case an attacker could open a merge request and run malicious code as a part of a workflow that gets executed by your self-hosted runner.

Properly managing mirrors can alleviate this problem.

Most importantly:

- A job launched on the self-hosted runner can access the whole machine/vm/container it is running on, and it runs in that context, so one might have to start the runner inside a container (but then, if a job itself requires to execute in a container, there might be issues)
- the workflow files might need to be adjusted, in particular the value associated to `runs-on` for every job (tags) need to be a subset of the ones that identify the self-hosted runner.

Other Material

Quick Reference

Glossary

linting

The act of running a *linter* on your source code, to find potential issues, like potential misuses/dangerous uses of language features, style violations and so on.

Available tools

Unit test frameworks

A **test framework** makes it easy to run tests across large amounts of code automatically. They provide more control than one single script which does some tests.

- Julia
- Python
  - Test
  - pytest
- Matlab
  - doctest
  - Class-Based Unit Tests
  - unittest
- C(++)
- R
  - Google Test
  - testthat
  - Catch2

- Julia
- Python
  - Test
  - pytest
- Matlab
  - doctest
  - Class-Based Unit Tests
  - unittest
- C(++)
- R
  - Google Test
  - testthat
  - Catch2
  - cppunit
  - Boost.Test
- Fortran
  - pFUnit
  - FRUIT
  - Ftunit

---

pytest

- Python
- https://docs.pytest.org/
- Installable via Conda or pip.
- Easy to use: Prefix a function with `test_` and the test runner will execute it. No need to subclass anything.

```
def get_word_lengths(s):
    """
    Returns a list of integers representing
    the word lengths in string s.
    """
    return [len(w) for w in s.split()]

def test_get_word_lengths():
    text = "Three tomatoes are walking down the street"
    assert get_word_lengths(text) == [5, 8, 3, 7, 4, 3, 6]
```

- Example project

---

testthat

- R
- https://github.com/r-lib/testthat
- Easily installed from CRAN with `install.packages("testthat")`, or from GitHub with `devtools::install_github("r-lib/testthat")`
- Use in package development with `usethis::use_testthat()`
- Add a new test file with `usethis::use_test("test-name")`, e.g.:

```
# tests/testthat/test_example.R
# file added by running `usethis::use_test("example")`

context("Arithmetics")
library("mypackage")

test_that("square root function works", {
  expect_equal(my_sqrt(4), 2)
  expect_warning(my_sqrt(-4))
})
```

```
# File added by running usethis::use_test("example")

context("Arithmetics")
library("mypackage")

test_that("square root function works", {
  expect_equal(my_sqrt(4), 2)
  expect_warning(my_sqrt(-4))
})

> devtools::test()
Loading mypackage
Testing mypackage
✔ |  OK F W S | Context
✔ |   2       | Arithmetics

══ Results ═══════════════════════════════════════════════════════════
OK:       2
Failed:   0
Warnings: 0
Skipped:  0
```

More information in the Testing chapter of the book R Packages by Hadley Wickham.

Test

- Julia
- - Documentation
- Part of the standard library
- Provides simple unit testing functionality with `@test` and `@test_throws` macros:

```
julia> using Test

julia> @test [1, 2] + [2, 1] == [3, 3]
Test Passed

# approximate comparisons:
julia> @test π ≈ 3.14 atol=0.01
Test Passed

# Tests that an expression throws exception:
julia> @test_throws BoundsError [1, 2, 3][4]
Test Passed
      Thrown: BoundsError

julia> @test_throws DimensionMismatch [1, 2, 3] + [1, 2]
Test Passed
      Thrown: DimensionMismatch
```

- Grouping related tests with the `@testset` macro:

```
using Test

function get_word_lengths(s::String)
    return [length(w) for w in split(s)]
end

@testset "Testing get_word_length()" begin
    text = "Three tomatoes are walking down the street"
    @test get_word_lengths(text) == [5, 8, 3, 7, 4, 3, 6]
```

```
using Test

function get_word_lengths(s::String)
    return [length(w) for w in split(s)]
end

@testset "Testing get_word_length()" begin
    text = "Three tomatoes are walking down the street"
    @test get_word_lengths(text) == [5, 8, 3, 7, 4, 3, 6]
    number = 123
    @test_throws MethodError get_word_lengths(number)
end
```

Catch2

- C++
- Project page with installation instructions
- Widely used
- Header-only
- Very rich in functionality
- Well-integrated with CMake

```
#include <catch2/catch.hpp>

#include "example.h"

using namespace Catch::literals;

TEST_CASE("Use the example library to add numbers", "[add]") {
  auto res = add_numbers(1.0, 2.0);
  REQUIRE(res == 3.0_a);
}
```

Google Test

- C++
- Documentation
- Widely used
- Very rich in functionality
- Well-integrated with CMake

```
#include <gtest/gtest.h>

#include "example.h"

TEST(example, add) {
  double res;
  res = add_numbers(1.0, 2.0);
  ASSERT_NEAR(res, 3.0, 1.0e-11);
}
```

- Documentation
- Very rich in functionality
- Header-only use possible

```
#include <boost/test/unit_test.hpp>
```

```
}
```

- Documentation
- Very rich in functionality
- Header only use possible

```
#include <boost/test/unit_test.hpp>

#include "example.h"

BOOST_AUTO_TEST_CASE( add )
{
  auto res = add_numbers(1.0, 2.0);
  BOOST_TEST(res == 3.0);
}
```

pFUnit

- Fortran
- Documentation and installation
- Very rich in functionality
- Requires modern Fortran compilers (uses F2003 standard)

```
@test
subroutine test_add_numbers()

   use hello
   use pfunit_mod

   implicit none

   real(8) :: res

   call add_numbers(1.0d0, 2.0d0, res)
   @assertEqual(res, 3.0d0)

end subroutine
```

- Example installation instructions

To test the `factorial` and `fizzbuzz` functions from the test-design exercises, use this `CMakeLists.txt` file:

```
cmake_minimum_required(VERSION 3.12)

project (PFUNIT_DEMO_CR
  VERSION 1.0.0
  LANGUAGES Fortran)

find_package(PFUNIT REQUIRED)
enable_testing()
```

```
cmake_minimum_required(VERSION 3.12)

project (PFUNIT_DEMO_CR
  VERSION 1.0.0
  LANGUAGES Fortran)

find_package(PFUNIT REQUIRED)
enable_testing()

# system under test
add_library (sut
  factorial.f90
  fizzbuzz.f90
  )

target_include_directories(sut PUBLIC ${CMAKE_CURRENT_BINARY_DIR})

# tests
set (test_srcs test_factorial.pf test_fizzbuzz.pf)
add_pfunit_ctest (my_tests
  TEST_SOURCES ${test_srcs}
  LINK_LIBRARIES sut
  )
```

You can then compile using this script:

```
#!/bin/bash -f

if [[ -d build ]]
then
    rm -rf build
fi

mkdir -p build
cd build
cmake .. -DCMAKE_PREFIX_PATH=$PFUNIT_DIR
make
./my_tests

# or
# ctest --verbose
```

- Example project

Services to deploy testing and coverage

Each of these are web services to handle testing, free for open source projects.

- GitHub Actions (we will demonstrate this in the next episode)
- GitLab CI (we will demonstrate this in the next episode)
- Azure Pipelines
- Coveralls

Keypoints

- Testing is a basic requirement of any possible language
- There are various tools for any language you may use
- There are free web services for open source

Good resources

- Getting Started With Testing in Python

List of exercises

Keypoints

- Testing is a basic requirement of any possible language
- There are various tools for any language you may use
- There are free web services for open source

Good resources

- Getting Started With Testing in Python

List of exercises

Full list

This is a list of all exercises and solutions in this lesson, mainly as a reference for helpers and instructors. This list is automatically generated from all of the other pages in the lesson. Any single teaching event will probably cover only a subset of these, depending on their interests.

Instructor guide

Detailed schedule

- 9:00-9:15 Motivation
- 9:15-9:40 Testing locally
  - explain the exercise: 5 min
  - **20 min exercise**
- 9:40-10:00 Automated testing
  - demo
- 10:00-10:10 Break
- 10:10-10:50 Test design
  - explain the exercise: 10 min
  - **20 min exercise**
  - discussion and type-along of advanced exercises: 10 minutes
- 10:50-11:00 Discussion and summary

Why we teach this lesson

- Because writing tests and using automated testing is often part of the development process, especially for codes that are larger than a simple script.
- We wish to give learners the tools so that they can rewrite codes and collaborate on a code projects with more confidence.
- Testing can help detecting regressions during development instead of later (when using the code) or too late (after having published results).
- Can simplify collaboration since contributors can be more confident that their changes preserve expected functionality.
- Tests document the intent of the function/module/library/code.
- Tests can guide the code towards more modular and reusable code style with fewer side effects.

Intended learning outcomes

- Be able to approach a function or module or library and design a test for it “in words” or pseudocode.
- Know that in each language tools exist that can execute a series of test functions and report success or failure.
- Know what test coverage means and how it can help to detect untested/unused code.
- Know that services like GitHub Actions, Azure pipelines, GitLab CI, Appveyor, and others allow us to automatically run tests on every push or every pull request or merge request.

How we teach this lesson

- Be able to approach a function or module or library and design a test for it “in words” or pseudocode.
- Know that in each language tools exist that can execute a series of test functions and report successes or failures.
- Know what test coverage means and how it can help to detect untested/unused code.
- Know that services like GitHub Actions, Azure Pipelines, GitLab CI, Appveyor, and others allow us to automatically run tests on every Git push or every pull request or merge request.
- Know where in a larger GitHub-hosted project to start introducing CI, as since it is beyond that and also probably not reasonable to test everything.

How we teach this lesson

How to start

The quote from another testing lesson by Kathryn Huff can be discussed to get participants to reflect on the role of software testing in science and its parallels to calibrating measurement devices. The second half of the quote is left out because it goes so far as to say that untested software does not constitute science, and this can be offending to learners.

Questions to involve participants

- Do you agree that testing software is similar to calibrating detectors?
- Do you test your code?
- How and when do you test?
- Where would you start adding a test in an existing project?
- In which situations would you recommend not to add any tests?

Structure of the lesson

Motivation and concepts

The former “motivation” and “concepts” episodes have been combined into one. Previously, the lesson was described as quite dogmagtic and without enough time for exercises. When teaching, try to give some of the benefits, but without being too long about it. The quick reference has more details on language-specific things.

Testing locally

After introducing the concepts the lesson becomes hands-on in the Testing Locally episode. The pytest example there can be done as type-along and the optional exercise “Testing with numerical tolerance” can be left as homework. The end-to-end test may be hard for many people who can’t script other programs, but it provides something that can keep advanced people interested for longer.

Automated testing (or full-cycle collaborative workflow)

After learning how to run tests locally, the lesson goes into automated testing using GitHub Actions or GitLab CI in the Automated Testing episode. This episode is a simplified version of the collaborative exercise in the optional episode “Full-cycle collaborative workflow”, and the instructor has a choice which one to teach (with the latter requiring around double the time). The exercise in Automated Testing can either be done as a type-along demonstration by the instructor or as an exercise in breakout rooms.

If the optional Full-cycle episode is covered, it is useful during the wrap up of the exercise to have a co-instructor doing the exercise in parallel with you. As your partner makes a pull requests with “fixes #1” (or whatever PR number) you can show that the PR links to a issue, and the issue pointing to a commit.

If run as an exercise, this episode requires that learners know how to set up Git repositories locally and online and how to work with pull/merge requests!

Test design

This episode now has exercises in different languages covering different testing approaches

If the optional Full-cycle episode is covered, it is useful during the wrap up of the exercise to have a co-instructor doing the exercise in parallel with you. As your partner makes a pull requests with "fixes #1" (or whatever PR number) you can show that the PR links to a issue, and the issue pointing to a commit.

If run as an exercise, this episode requires that learners know how to set up Git repositories locally and online and how to work with pull/merge requests!

Test design

This episode now has exercises in different languages covering different testing approaches (unit tests, integration tests, testing randomness, TDD). In normal workshops there are far too many exercises for the available time so the instructor has freedom to recommend specific exercises, to let learners choose themselves, or to recommend learners to discuss rather than implement the tests.

Order of Continuous Integration and Test Design

When this lesson is taught standalone in a software testing workshop or hackathon, it might be better to go through the Test Design episode before the Continuous Integration episode.

Timing

The lesson is stipulated to take around 2 hours, but if more time is available you can replace the Automated Testing episode with "Full-cycle collaborative workflow", or give more time to the Test Design episode.

Splitting the lesson in two halves with a lunch break in the middle works well. Before lunch you have on hour introduction and the two first exercises. After lunch you continue with the automated testing exercise and test design.

Room for improvement

- The lesson is still too Python centric in the "Testing locally" and "Automated testing" episodes. It would be nice to offer alternatives for learners who write in different languages.
- Currently the lesson is in the "discipline" domain. Testing is something you do to control behavior (your own as well as the code). Testing is also something you can do to improve productivity, but this is less visible/detectable.

Field reports

2024 March (first time with no exercises)

This was the first year we did the "all demo" mode with no exercises, and it worked well enough. Our rough plan can be seen at this hackmd (not pasted here because it's kind of long).

We tried to be fast with motivation, and not go to details. In "testing locally", we basically did the exercise as a demo. There wasn't much of note, it worked and there was plenty of discussion. "Automated testing" worked fine also, except: a) we had to choose how to push to Github, we used vscode, but not all learners did that method. Nothing to change here, but be aware you should explain what and why you are doing for people who didn't use your chosen method of linking, and b) code coverage didn't work from a PR from a fork. We had one person set up the automated tests, then swapped instructors and showed the second making the fork and contributing.

The biggest thing to think about is how, when demoing testing locally/automated testing, people ask "how to tests work?", which then comes later. I don't know if I would change the order, though - because showing how to design tests without knowing how to use them would make the same kind of questions, but reversed. I think it's just a "decide how it is and prepare for the questions" - maybe in "motivation" or some other summary episode at the start.

The biggest thing to think about is how, when demoing testing locally/automated testing, be aware you should explain what and why you are doing for people who didn't use your chosen method of linking, and b) code coverage didn't work from a PR from a fork. We had people ask "how to tests work?", which then comes later. I don't know if I would change the order, though... because showing how to design tests without knowing how to use them one person set up the automated tests, then swapped instructors and showed the second would make the same kind of questions, but reversed. I think it's just a "decide how it is and making the fork and contributing.
prepare for the questions" - maybe in "motivation" or some other summary episode at the start.

"Test design" didn't have any major problems, but many of the tests were relatively simple to discuss: the earlier tests would be trivial to understand if someone shows you the code (should we demonstrate typing two lines that's the same as in motivation? - but interesting to do as an exercise). Some of the later ones were interesting to build up and demonstrate. It would be good to think which exercises one wants to demo, and which one wants to only talk about. And think which comes out of each demo/telling.

Overall, no major issues in doing this demo-based with good familiarity but limited preparation time.

2023 September

For this lesson, "Motivation" and "Concepts" were combined. An extra end-to-end test was added under "testing locally".

2023 March

Due to a mix-up with instructors, we changed the plan at the last minute and it didn't go so well (we should have known). The original plan had pytest as a demo, with exercises for Github CI and test design. We did pytest as an exercise, Github CI as an exercise, and had almost no time for test design. One complaint was that there wasn't enough time for practical things, so I think a plan like last time could have been good. As usual, we should have found a way to talk less and do more.

This was the schedule from March 2023:

```
* 8:50 - 9:00 Getting started
* 9:00 - 10:45 [Software testing](https://coderefinery.github.io/testing/)
  - 9:00-9:05 Short info about today's exercise sessions and possible questions from
yesterday
  - 9:05-9:10 [Motivation](https://coderefinery.github.io/testing/motivation/)
  - 9:10-9:20 [Concepts](https://coderefinery.github.io/testing/concepts/)
  - 9:20-9:45 [Testing locally](https://coderefinery.github.io/testing/pytest/)
      - 5 minutes talking
      - 15 minute exercises
      - 5 going over exercises and discussion
  - 9:45-9:55 Break
  - 9:55-10:20 [Automated testing](https://coderefinery.github.io/testing/continuous-
integration/)
      - type-along session
  - 10:20-10:45 [Test design](https://coderefinery.github.io/testing/test-design/)
      - discussion: 5 minutes
      - Exercises: 10 minutes.  Any of exercises Design-1 to Design-8 that learners
want to do.
      - discussion and type-along of advanced exercises: 10 minutes
  - 10:45-11:00 Break
```

(pytest locally), which I would agree with. I think for next time, we should add in parts of that exercise so that there are more advanced things to do, since there are more nice features of pytest, such as `--pdb`, running single tests, etc. Other than that, there wasn't really much to report.

2022 September

```
      - discussion and type-along of advanced exercises: 10 minutes
  - 10:45-11:00 Break
```

(pytest locally), which I would agree with. I think for next time, we should add in parts of that exercise so that there are more advanced things to do, since there are more nice features of pytest, such as `--pdb`, running single tests, etc. Other than that, there wasn't really much to report.

The way extra long breaks and a 5-minute starting point were included is quite good for the schedule. Not because we needed them, but it allowed us to go over time and not be under so much time pressure.

```
**Day 6**
* 8:50 - 9:00 Getting started
* 9:00 - 10:45 Software testing
  - 9:00-9:05 Short info about today's exercise sessions and possible questions from
yesterday
  - 9:05-9:10 [Motivation](https://coderefinery.github.io/testing/motivation/)
  - 9:10-9:20 [Concepts](https://coderefinery.github.io/testing/concepts/)
  - 9:20-9:45 [Testing locally](https://coderefinery.github.io/testing/pytest/)
      - 5 minutes talking
      - 15 minute exercises
      - 5 going over exercises and discussion
  - 9:45-9:55 Break
  - 9:55-10:20 [Automated testing](https://coderefinery.github.io/testing/continuous-
integration/)
      - type-along session
  - 10:20-10:45 [Test design](https://coderefinery.github.io/testing/test-design/)
      - discussion: 5 minutes
      - Exercises: 10 minutes.  Any of exercises Design-1 to Design-8 that learners
want to do.
      - discussion and type-along of advanced exercises: 10 minutes
  - 10:45-11:00 Break
```

2019 November instructor training workshop

Some notes about teaching are here: https://github.com/coderefinery/testing/issues/42

Credit and license

This material is provided by CodeRefinery under the licenses stated below.

Website template

The website template is maintained by CodeRefinery and rendered with sphinx-lesson: structured lessons with Sphinx.

Instructional material

All CodeRefinery instructional material is made available under the Creative Commons Attribution license (CC-BY-4.0). The following is a human-readable summary of (and not a substitute for) the full legal text of the CC-BY-4.0 license.

You are free:

- to **Share** - copy and redistribute the material in any medium or format
- to **Adapt** - remix, transform, and build upon the material

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

- **Attribution** - You must give appropriate credit (mentioning that your work is derived from work that is Copyright (c) CodeRefinery and, where practical, linking to https://coderefinery.org, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

You are free:

- to **Share** - copy and redistribute the material in any medium or format
- to **Adapt** - remix, transform, and build upon the material

Attribution - You must give appropriate credit (mentioning that your work is derived from work that is Copyright (c) CodeRefinery and, where practical, linking to https://coderefinery.org, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions - You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits. With the understanding that:

- You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.
- No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.